

**LAPORAN AKHIR PRAKTIKUM
ALGORITMA & STRUKTUR DATA**



Oleh:

Amalia Soraya

NIM. 2510817120002

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
2026**

LEMBAR PENGESAHAN

LAPORAN AKHIR PRAKTIKUM ALGORITMA & STRUKTUR

DATA

Laporan Praktikum Algoritma & Struktur Data

Modul 1 : Struct dan Pointer

Modul 2 : Stack dan Queue

Modul 3 : Linked List

Modul 4 : Sorting

Modul 5 : Searching

Modul 6 : Graph dan Tree

ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma dan Struktur Data. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Amalia Soraya

NIM : 2510817120002

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
DAFTAR ISI	ii
DAFTAR GAMBAR.....	vii
DAFTAR TABEL	viii
MODUL 1: STRUCT DAN POINTER	10
SOAL 1.....	10
A. Source Code.....	10
B. Output Program	11
C. Pembahasan	12
SOAL 2.....	14
A. Source Code.....	14
B. Output Program	15
C. Pembahasan	15
MODUL 2: STACK DAN QUEUE	18
SOAL 1.....	18
A. Source Code.....	18
B. Pembahasan	19
SOAL 2.....	21
A. Source Code.....	22
B. Output Program	23
C. Pembahasan	24
SOAL 3.....	25
A. Source Code.....	25
B. Pembahasan	26

SOAL 4.....	28
A. Source Code.....	29
B. Output Program	30
C. Pembahasan	30
MODUL 3: LINKED LIST	32
SOAL 1.....	32
A. Source Code.....	32
B. Pembahasan	36
SOAL 2.....	40
A. Source Code.....	41
B. Output Program	42
C. Pembahasan	42
SOAL 3.....	44
A. Source Code.....	44
B. Pembahasan	48
SOAL 4.....	52
A. Source Code.....	53
B. Output Program	55
C. Pembahasan	55
MODUL 4: SORTING	57
BUBBLE SORT	57
A. Source Code.....	57
B. Output Program	57
C. Pembahasan	58
SELECTION SORT	59

A. Source Code.....	59
B. Output Program	59
C. Pembahasan	59
INSERTION SORT	61
A. Source Code.....	61
B. Output Program	61
C. Pembahasan	62
MERGE SORT	63
A. Source Code.....	63
B. Output Program	64
C. Pembahasan	64
QUICK SORT	66
A. Source Code.....	66
B. Output Program	67
C. Pembahasan	67
RADIX SORT	69
A. Source Code.....	69
B. Output Program	70
C. Pembahasan	70
ANALISIS PERFORMA SORTING	72
MODUL 5: SEARCHING	73
LINEAR SEARCH.....	73
A. Source Code.....	73
B. Output Program	74
C. Pembahasan	74

BINARY SEARCH	76
A. Source Code.....	76
B. Output Program	77
C. Pembahasan	77
ANALISIS PERFORMA SEARCHING	79
MODUL 6: GRAPH DAN TREE	80
SOAL 1	80
A. Source Code.....	82
B. Output Program	84
C. Pembahasan	84
SOAL 2	86
A. Source Code.....	87
B. Output Program	89
C. Pembahasan	89
SOAL 3	90
A. Source Code.....	92
B. Output Program	93
C. Pembahasan	93
SOAL 4	95
A. Source Code.....	97
B. Output Program	98
C. Pembahasan	98
SOAL 5	100
A. Source Code.....	101
B. Output Program	103

C. Pembahasan	104
TAUTAN GIT	105

DAFTAR GAMBAR

MODUL 1: STRUCT DAN POINTER

Gambar 1. Screenshot Jawaban Soal 1 Modul 1	11
Gambar 2. Screenshot Jawaban Soal 2 Modul 1	15

MODUL 2: STACK DAN QUEUE

Gambar 3. Screenshot Jawaban Soal 2 Modul 2	23
Gambar 4. Screenshot Jawaban Soal 4 Modul 2	30

MODUL 3: LINKED LIST

Gambar 5. Screenshot Jawaban Soal 2 Modul 3	42
Gambar 6. Screenshot Jawaban Soal 4 Modul 3	55

MODUL 4: SORTING

Gambar 7. Screenshot Jawaban Bubble Sort.....	57
Gambar 8. Screenshot Jawaban Selection Sort	59
Gambar 9. Screenshot Jawaban Insertion Sort	61
Gambar 10. Screenshot Jawaban Merge Sort.....	64
Gambar 11. Screenshot Jawaban Quick Sort	67
Gambar 12. Screenshot Jawaban Radix Sort.....	70

MODUL 5: SEARCHING

Gambar 13. Screenshot Jawaban Linear Search.....	74
Gambar 14. Screenshot Jawaban Binary Search	77

MODUL 6: GRAPH DAN TREE

Gambar 15. Screenshot Jawaban Soal 1 Modul 6	84
Gambar 16. Screenshot Jawaban Soal 2 Modul 6	89
Gambar 17. Screenshot Jawaban Soal 3 Modul 6	93
Gambar 18. Screenshot Jawaban Soal 4 Modul 6	98
Gambar 19. Screenshot Jawaban Soal 5 Modul 6	103

DAFTAR TABEL

MODUL 1: STRUCT DAN POINTER

Tabel 1. Source Code File line.cpp	10
Tabel 2. Source Code File prob1.cpp	11
Tabel 3. Source Code File circle.cpp.....	14
Tabel 4. Source Code File prob2.cpp	14

MODUL 2: STACK DAN QUEUE

Tabel 5. Source Code File stack.cpp	18
Tabel 6. Source Code File prob1.cpp	22
Tabel 7. Source Code File queue.cpp	25
Tabel 8. Source Code File prob2.cpp	29

MODUL 3: LINKED LIST

Tabel 9. Source Code File single_linked_list.cpp	32
Tabel 10. Source Code File prob_a.cpp	41
Tabel 11. Source Code File double_linked_list.cpp.....	44
Tabel 12. Source Code File prob_b.cpp	53

MODUL 4: SORTING

Tabel 13. Source Code Bubble Sort	57
Tabel 14. Source Code Selection Sort	59
Tabel 15. Source Code Insertion Sort.....	61
Tabel 16. Source Code Merge Sort	63
Tabel 17. Source Code Quick Sort	66
Tabel 18. Source Code Radix Sort	69

MODUL 5: SEARCHING

Tabel 19. Source Code Linear Search	73
Tabel 20. Source Code Binary Search.....	76

MODUL 6: GRAPH DAN TREE

Tabel 21. Source Code File soal_1.cpp	82
Tabel 22. Source Code File soal_2.cpp	87
Tabel 23. Source Code File soal_3.cpp	92
Tabel 24. Source Code File soal_4.cpp	97

Tabel 25. Source Code File soal_5.cpp	101
---	-----

MODUL 1:

STRUCT DAN POINTER

SOAL 1

Diberikan sebuah garis l yang direpresentasikan dalam bentuk persamaan umum:

$$ax + by + c = 0$$

dan sebuah titik $P = (x_p, y_p)$. Tentukan posisi titik P terhadap garis l dengan cara mengimplementasikan fungsi `gradient` dan `CheckPointPosition` pada file header yang telah diberikan.

Input	Output
1 0 -3 5 0	Left
1 0 -3 1 7	Right
2 -1 -1 1 1	On Line

A. Source Code

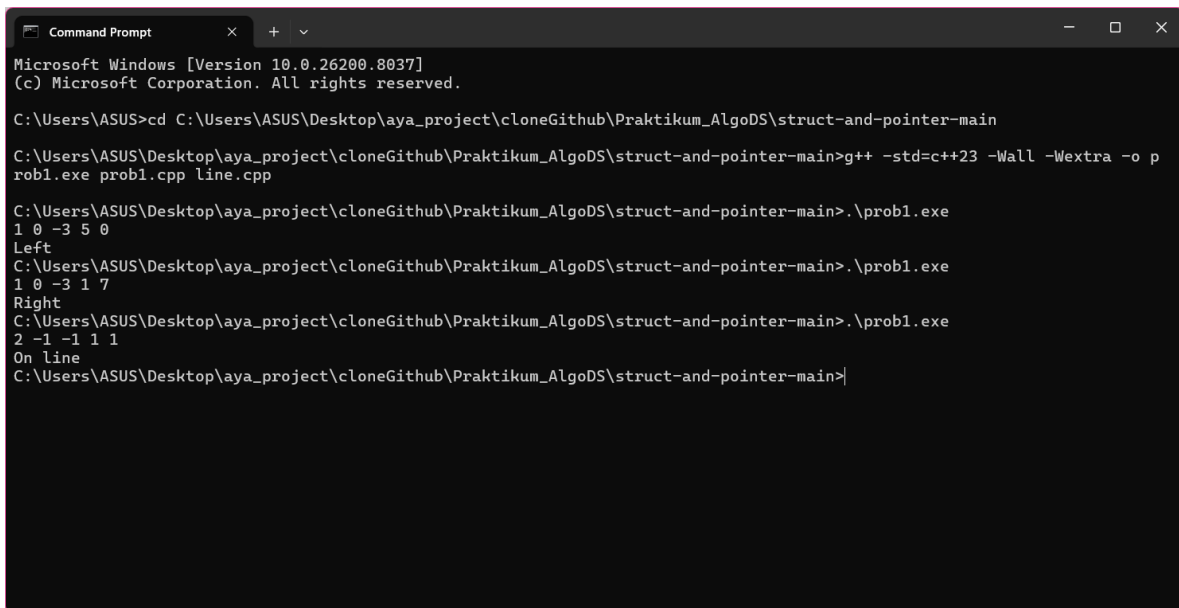
Tabel 1. Source Code File `line.cpp`

```
1  #include "line.h"
2  #include "point.h"
3  #include <string>
4
5  double gradient(const Line * l, const Point * p) {
6
7      return (l -> a * p -> x) + (l -> b * p -> y) + l ->
c;
8  }
9
10 std::string CheckPointPosition(double gradient) {
11
12     if(gradient == 0) {
13         return "On line";
14     } else if (gradient > 0) {
15         return "Left";
16     } else{
17         return "Right";
18     }
19 }
```

Tabel 2. Source Code File prob1.cpp

```
1  #include <iostream>
2  using namespace std;
3  #include "point.h"
4  #include "line.h"
5  #include <string>
6
7  int main() {
8      Line l;
9      Point p;
10
11      cin >> l.a >> l.b >> l.c >> p.x >> p.y;
12
13      double result = gradient(&l, &p);
14      cout << CheckPointPosition(result);
15
16 }
```

B. Output Program



```
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ASUS>cd C:\Users\ASUS\Desktop\aya_project\cloneGithub\Praktikum_AlgoDS\struct-and-pointer-main

C:\Users\ASUS\Desktop\aya_project\cloneGithub\Praktikum_AlgoDS\struct-and-pointer-main>g++ -std=c++23 -Wall -Wextra -o prob1.exe prob1.cpp line.cpp

C:\Users\ASUS\Desktop\aya_project\cloneGithub\Praktikum_AlgoDS\struct-and-pointer-main>.\prob1.exe
1 0 -3 5 0
Left
C:\Users\ASUS\Desktop\aya_project\cloneGithub\Praktikum_AlgoDS\struct-and-pointer-main>.\prob1.exe
1 0 -3 1 7
Right
C:\Users\ASUS\Desktop\aya_project\cloneGithub\Praktikum_AlgoDS\struct-and-pointer-main>.\prob1.exe
2 -1 -1 1 1
On line
C:\Users\ASUS\Desktop\aya_project\cloneGithub\Praktikum_AlgoDS\struct-and-pointer-main>
```

Gambar 1. Screenshot Jawaban Soal 1 Modul 1

C. Pembahasan

1. Implementasi line

Pada baris 1 sampai 3 adalah pemanggilan library. Gunanya agar kode di file ini dapat menggunakan fungsi yang tersedia di `line.h`, `point.h`, dan `string`.

Pada baris 5 sampai 8 adalah fungsi `gradient`. Gunanya untuk menghitung nilai gradien suatu titik terhadap garis (buat menentukan posisi titik tersebut relatif ke garis). Cara kerjanya yaitu memasukkan nilai koefisien garis (a , b , c) dan koordinat titik (x , y) ke dalam rumus persamaan garis $ax + by + c$, lalu hasilnya langsung dikembalikan (`return`). Kompleksitas waktu fungsi ini adalah $O(1)$ karena hanya ada operasi aritmatika, tanpa loop yang bergantung ke nilai N . Kompleksitas ruang fungsi ini adalah $O(1)$ karena tidak ada variabel tambahan yang bergantung pada nilai N hasil input.

Pada baris 10 sampai 19 adalah fungsi `CheckPointPosition`. Gunanya untuk menentukan posisi titik terhadap garis berdasarkan nilai gradiennya. Cara kerjanya dengan mengecek dulu apakah nilai `gradient` sama dengan 0? Kalau iya, berarti titik ada tepat di garis, `return "On line"`. Kalau gradiennya lebih dari 0, berarti titik ada di sisi kiri garis, `return "Left"`. Kalau tidak keduanya (berarti kurang dari 0), titik ada di sisi kanan garis, `return "Right"`. Kompleksitas waktunya $O(1)$ karna hanya serangkaian percabangan `if-else` dan kompleksitas ruangnya $O(1)$ karna tidak ada variabel tambahan yang dipakai.

2. Problem

Pada baris 1 sampai 5 adalah pemanggilan library. Gunanya agar program dapat menggunakan fungsi yang tersedia di `iostream`, `point.h`, `line.h`, dan `string`. Baris 2 gunanya efisiensi pengetikan.

Pada baris 8 dan 9 adalah deklarasi struct `Line` dengan nama `l` dan struct `Point` dengan nama `p`. Gunanya untuk mendeklarasi variabel garis dan titik yang akan dipakai dalam program.

Baris 11 digunakan untuk input user, cara kerjanya dengan meminta input nilai dari user secara berurutan, lalu hasil input disimpan ke koefisien garis (`l.a`, `l.b`, `l.c`) dan koordinat titik (`p.x`, `p.y`).

Baris 13 digunakan untuk memanggil fungsi `gradient`, cara kerjanya fungsi diisi argumen alamat `l` dan alamat `p`, lalu hasil perhitungan gradiennya disimpan ke variabel `result`.

Baris 14 digunakan untuk menampilkan posisi titik terhadap garis, cara kerjanya dengan memanggil fungsi `CheckPointPosition` dan diisi argumen variabel `result`, lalu hasilnya (string posisi) langsung dicetak ke output.

Kompleksitas ruang program ini adalah $O(1)$ karena hanya ada beberapa variabel konstan (`struct Line`, `Point`, dan `result`) yang tidak bergantung pada ukuran N dari input. Kompleksitas waktunya juga $O(1)$ karena hanya ada input, satu kali pemanggilan fungsi `gradient` dan `CheckPointPosition` yang masing-masing $O(1)$, tanpa ada loop sama sekali.

SOAL 2

Diberikan sebuah lingkaran C dengan pusat (c_x, c_y) dan jari-jari $r > 0$, serta sebuah titik $P = (p_x, p_y)$. Tentukan posisi titik P terhadap lingkaran C dan mengimplementasikan fungsi `distance` dan `CheckPointInCircle` pada file header yang diberikan.

Input	Output
0 0 5 3 4	On Circle
0 0 5 1 1	Inside
0 0 5 4 4	Outside
-2 -2 4 2 -2	On Circle

A. Source Code

Tabel 3. Source Code File circle.cpp

1	#include "circle.h"
2	#include "point.h"
3	#include <cmath>
4	
5	double distance(const Circle * c, const Point * p) {
6	return sqrt(pow((p -> x - c -> centre.x), 2) + pow((p
7	-> y - c -> centre.y), 2));
8	}
9	
10	std::string CheckPointPosition(double distance, const
11	Circle * c) {
12	if(distance == pow(c -> radius, 2)) {
13	return "On Circle";
14	} else if (distance < pow(c -> radius, 2)) {
15	return "Inside";
16	} else{
17	return "Outside";
18	}
19	}

Tabel 4. Source Code File prob2.cpp

1	#include <iostream>
2	using namespace std;
3	#include <string>
4	#include <cmath>

```

5  #include "circle.h"
6  #include "point.h"
7
8  int main() {
9      Point p;
10     Circle c;
11
12     cin >> c.centre.x >> c.centre.y >> c.radius >> p.x
    >> p.y;
13
14     double distances = distance(&c, &p);
15     cout << CheckPointInCircle(distances, &c);
16
17 }

```

B. Output Program

```

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ASUS>cd C:\Users\ASUS\Desktop\aya_project\cloneGithub\Praktikum_AlgoDS\struct-and-pointer-main

C:\Users\ASUS\Desktop\aya_project\cloneGithub\Praktikum_AlgoDS\struct-and-pointer-main>.\prob2.exe
0 0 5 3 4
On Circle
C:\Users\ASUS\Desktop\aya_project\cloneGithub\Praktikum_AlgoDS\struct-and-pointer-main>.\prob2.exe
0 0 5 1 1
Inside
C:\Users\ASUS\Desktop\aya_project\cloneGithub\Praktikum_AlgoDS\struct-and-pointer-main>.\prob2.exe
0 0 5 4 4
Outside
C:\Users\ASUS\Desktop\aya_project\cloneGithub\Praktikum_AlgoDS\struct-and-pointer-main>.\prob2.exe
-2 -2 4 2 -2
On Circle
C:\Users\ASUS\Desktop\aya_project\cloneGithub\Praktikum_AlgoDS\struct-and-pointer-main>

```

Gambar 2. Screenshot Jawaban Soal 2 Modul 1

C. Pembahasan

1. Implementasi circle

Pada baris 1 sampai 3 adalah pemanggilan library. Gunanya agar kode di file ini dapat menggunakan fungsi yang tersedia di `circle.h`, `point.h`, dan `cmath`.

Pada baris 5 sampai 8 adalah fungsi `distance`. Gunanya untuk menghitung jarak antara suatu titik dengan titik pusat lingkaran. Cara kerjanya yaitu selisih

koordinat x titik dengan x pusat lingkaran dikuadratkan, ditambah selisih koordinat y titik dengan y pusat lingkaran yang juga dikuadratkan, lalu hasil penjumlahan itu diakarkan (`sqrt`) sesuai rumus jarak, lalu return hasil. Kompleksitas waktu fungsi ini adalah $O(1)$ karena hanya operasi aritmatika tanpa loop. Kompleksitas ruang fungsi ini adalah $O(1)$ karena tidak ada variabel tambahan yang bergantung pada ukuran input.

Pada baris 10 sampai 19 adalah fungsi `CheckPointPosition`. Gunanya untuk menentukan posisi titik terhadap lingkaran berdasarkan nilai `distance` yang sudah dihitung. Cara kerjanya dengan mengecek dulu apakah nilai `distance` sama dengan radius lingkaran yang dikuadratkan (`pow(c->radius, 2)`)? Kalau iya, berarti titik ada tepat di garis lingkaran, return "On Circle". Kalau `distance`-nya kurang dari radius kuadrat, berarti titik ada di dalam lingkaran, return "Inside". Kalau tidak keduanya (berarti lebih besar), titik ada di luar lingkaran, return "Outside". Kompleksitas waktunya $O(1)$ karna serangkaian percabangan if-else tanpa loop dan kompleksitas ruangnya $O(1)$ karna tidak ada variabel tambahan yang dipakai.

2. Problem

Pada baris 1 sampai 6 adalah pemanggilan library. Gunanya agar program ini dapat menggunakan fungsi yang tersedia di `iostream`, `string`, `cmath`, `circle.h`, dan `point.h`. Baris 2 gunanya efisiensi pengetikan.

Pada baris 9 dan 10 adalah deklarasi struct `Point` dengan nama `p` dan struct `Circle` dengan nama `c`. Gunanya untuk mendeklarasi variabel titik dan lingkaran yang akan dipakai dalam program.

Baris 12 digunakan untuk input user, cara kerjanya dengan meminta input nilai dari user secara berurutan, lalu hasil input disimpan ke titik pusat dan radius lingkaran (`c.centre.x`, `c.centre.y`, `c.radius`) dan koordinat titik (`p.x`, `p.y`).

Baris 14 digunakan untuk memanggil fungsi `distance`, cara kerjanya fungsi diisi argumen alamat `c` dan alamat `p`, lalu hasil perhitungan jaraknya disimpan ke variabel `distances`.

Baris 15 digunakan untuk menampilkan posisi titik terhadap lingkaran, cara kerjanya dengan memanggil fungsi `CheckPointInCircle` dan diisi argumen variabel `distances` dan alamat `c`, lalu hasilnya langsung dicetak ke output.

Kompleksitas ruang program ini adalah $O(1)$ karena hanya ada beberapa variabel konstan (`struct Point`, `Circle`, dan `distances`) yang tidak bergantung pada ukuran input N . Kompleksitas waktunya juga $O(1)$ karena hanya ada input, satu kali pemanggilan fungsi `distance` dan `CheckPointInCircle` yang masing-masing $O(1)$, tanpa loop.

MODUL 2:

STACK DAN QUEUE

SOAL 1

Pada file stack.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file stack.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 5. Source Code File stack.cpp

1	#include "stack.h"
2	#include <stdexcept>
3	
4	void init(Stack* s) {
5	s->top = s->data ;
6	}
7	bool isEmpty(const Stack* s) {
8	return s->top == s->data;
9	}
10	bool isFull(const Stack* s) {
11	return s->top == (s->data + (MAX - 1));
12	}
13	void push(Stack* s, int value) {
14	if (!isFull(s)) {
15	s->top++;
16	*s->top = value;
17	}
18	throw std::runtime_error("Stack kosong");
19	}
20	void pop(Stack* s) {
21	if (!isEmpty(s)) {
22	s->top--;
23	}
24	throw std::runtime_error("Stack kosong");
25	}
26	int peek(const Stack* s) {
27	if (!isEmpty(s)) {

28	return *(s->top);
29	}
30	throw std::runtime_error("Stack kosong");
31	}

B. Pembahasan

Pada baris 3 sampai 5 adalah fungsi `init`. Gunanya untuk inisialisasi stack agar siap dipakai. Cara kerjanya yaitu pointer `top` diarahkan ke alamat data (awal array), jadi posisi `top` sama dengan posisi data. Kompleksitas waktu fungsi ini adalah $O(1)$ karena hanya assignment tanpa loop. Kompleksitas ruang fungsi ini adalah $O(1)$ karena tidak ada variabel yang bergantung pada ukuran array.

Pada baris 6 sampai 8 adalah fungsi `isEmpty`. Gunanya untuk mengecek apakah stack kosong atau tidak. Cara kerjanya dengan mengecek apakah `top` menunjuk ke alamat yang sama dengan data? Jika iya, berarti stack kosong, nilainya `true`, kalau tidak, `false`. Kompleksitas waktunya $O(1)$ karna hanya perbandingan dan kompleksitas ruangnya $O(1)$ karna tidak ada variabel tambahan.

Pada baris 9 sampai 11 adalah fungsi `isFull`. Gunanya untuk mengecek apakah stack udah penuh atau belum. Cara kerjanya dengan mengecek apakah `top` udah menunjuk ke alamat data ditambah ($MAX-1$), yang berarti sudah sampai elemen paling belakang dari kapasitas stack. Kalau iya, stack penuh, kalau tidak, belum penuh. Kompleksitas waktunya $O(1)$ karna perbandingan dan kompleksitas ruangnya $O(1)$ karna tidak ada variabel tambahan.

Pada baris 12 sampai 17 adalah fungsi `push`. Gunanya untuk menambahkan nilai baru ke stack. Cara kerjanya pertama ngecek dulu apakah stack sudah penuh atau belum pakai fungsi `isFull`. Kalau belum penuh, `top` digeser maju satu posisi dulu (`top++`), baru nilai yang diinput dimasukin ke posisi `top` yang baru itu. Kalau udah penuh, tidak terjadi apa-apa (tidak nambah). Kompleksitas waktunya $O(1)$ karna tidak ada loop dan kompleksitas ruangnya $O(1)$ karna cuma nambah 1 elemen aja.

Pada baris 18 sampai 22 adalah fungsi `pop`. Gunanya untuk menghapus nilai paling atas dari stack. Cara kerjanya pertama ngecek dulu apakah stack kosong atau tidak dengan fungsi `isEmpty`. Kalau tidak kosong, `top` digeser mundur satu posisi (`top--`), jadi nilai sebelumnya dianggap sudah terhapus walau datanya masih ada di memori. Kompleksitas

waktunya $O(1)$ karna tidak ada proses loop yang memengaruhi dan kompleksitas ruangnya $O(1)$ karna tidak nambah variabel apapun.

Pada baris 23 sampai 28 adalah fungsi `peek`. Gunanya untuk mengakses nilai paling atas stack tanpa menghapusnya. Cara kerjanya pertama ngecek dulu apakah stack kosong atau tidak. Kalau tidak kosong, kembalikan nilai yang ditunjuk oleh `top`. Kalau kosong, throw pesan error. Kompleksitas waktunya $O(1)$ karna hanya pengecekan dan return, kompleksitas ruangnya $O(1)$ karna tidak ada variabel tambahan.

SOAL 2

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut,
- lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n adalah jumlah simbol dalam ekspresi
- A_i adalah nilai bilangan pada input
- Simbol yang valid hanya:
 - bilangan bulat
 - operator: +, -, *, /

Format Input

n

$A_1 A_2 \dots A_n$

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
3 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Tabel 6. Source Code File probl.cpp

1	#include "stack.h"
2	#include <iostream>
3	using namespace std;
4	#include <string>
5	
6	int main() {
7	Stack s;
8	init(&s);
9	int N, angka, a, b, hasil;
10	string x;
11	
12	cin >> N;
13	for (int i = 0; i < N; i++) {
14	cin >> x;
15	
16	if (x != "+" && x != "-" && x != "*" && x !=
17	"/") {
18	angka = stoi(x);
19	push(&s, angka);
20	} else {
21	if (isEmpty(&s)) {
22	cout << "Stack masih kosong" << endl;

```

23         return 1;
24     }
25     a = peek(&s);
26     pop(&s);
27
28     if (isEmpty(&s)) {
29         cout << "Stack masih kosong" << endl;
30         return 1;
31     }
32     b = peek(&s);
33     pop(&s);
34
35     if (x == "+") hasil = b + a;
36     else if (x == "-") hasil = b - a;
37     else if (x == "*") hasil = b * a;
38     else if (x == "/") {
39         if (a == 0){
40             cout << "Gabisa dibagi 0" << endl;
41         }
42         hasil = b / a;
43     }
44     push(&s, hasil);
45 }
46 }
47 cout << peek(&s) << endl;
48 // cout << a << " " << b << " " << hasil << endl;
49 return 0;
50 }

```

B. Output Program

```

PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1> g++ -std=c++23 -Wall -Wext
-ra -o prob1.exe prob1.cpp stack.cpp
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1> .\prob1.exe
3
2 3 +
5
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1> .\prob1.exe
9
5 1 2 + 4 * + 3 -
14
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1> .\prob1.exe
5
10 2 / 3 *
15
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1>

```

Gambar 3. Screenshot Jawaban Soal 2 Modul 2

C. Pembahasan

Pada baris 1 sampai 4 adalah pemanggilan library. Gunanya agar kode di file ini dapat menggunakan fungsi yang tersedia di `stack.h`, `iostream`, dan `string`. Baris 3 gunanya efisiensi pengetikan.

Pada baris 7 dan 8 adalah deklarasi struct `Stack` dengan nama `s`, lalu dipanggil fungsi `init`. Gunanya untuk mendeklarasi struct agar bisa dipakai sebagai stack dalam program, dan `init` dipanggil untuk menyiapkan pointer `top`.

Pada baris 9 dan 10 digunakan untuk deklarasi variabel yang akan dipakai dalam program. Variabel `N` untuk jumlah token, `angka`, `a`, `b`, `hasil` untuk kalkulasi, dan `x` bertipe string untuk nampung input user (angka atau operator).

Baris 12 digunakan untuk input user, cara kerjanya dengan meminta input nilai dari user, lalu hasil input disimpan ke variabel `N` (jumlah yang akan diproses).

Pada baris 13 sampai 46 adalah looping utama program, digunakan untuk membaca token satu-satu dan memproses notasi pakai stack. Cara kerjanya yaitu program iterasi sebanyak `N` kali, setiap iterasi user input satu token ke variabel `x`. Lalu dicek apakah `x` itu bukan salah satu dari `"+"`, `"-"`, `"*"`, `"/"`? Kalau bukan operator (berarti angka), `x` diconvert ke `int(stoi)` disimpan ke `angka`, lalu di-push ke stack. Kalau `x` adalah operator, masuk ke `else`: pertama cek dulu apakah stack kosong, kalau kosong cetak pesan error dan langsung `return 1` (program berhenti). Kalau tidak kosong, ambil nilai teratas dengan `peek` ke variabel `a`, lalu `pop`. Dicek lagi apakah stack kosong, kalau kosong cetak error dan `return 1`. Kalau tidak kosong, ambil nilai teratas lagi ke variabel `b`, lalu `pop`. Setelah dapat dua operand (`b` dan `a`), hasil dihitung sesuai operatornya: kalau `"+"` maka `b+a`, kalau `"-"` maka `b-a`, kalau `"*"` maka `b*a`, kalau `"/"` maka `b/a` (dengan pengecekan dulu kalau `a` sama dengan 0, cetak pesan "Gabisa dibagi 0"). Terakhir, hasil kalkulasi di-push lagi ke stack buat dipakai di operasi selanjutnya.

Baris 47 digunakan untuk menampilkan nilai akhir yang tersisa di puncak stack, yaitu hasil akhir dari postfix tersebut.

Kompleksitas ruang program ini adalah $O(n)$ karena ada stack yang menyimpan banyak nilai sebanyak token yang di-push. Kompleksitas waktunya $O(n)$ karena ada satu loop yang jalan sebanyak `N` kali, dan di dalamnya fungsi `push/pop/peek/isEmpty` semuanya $O(1)$ sehingga tidak menambah kompleksitas.

SOAL 3

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 7. Source Code File queue.cpp

1	#include "queue.h"
2	#include <stdexcept>
3	
4	void init(Queue* q) {
5	q->front = q->data;
6	q->rear = q->data - 1;
7	}
8	
9	bool isEmpty(const Queue* q) {
10	return q->front > q->rear;
11	}
12	
13	bool isFull(const Queue* q) {
14	return q->rear == (q->data + (MAX - 1));
15	}
16	
17	void enqueue(Queue* q, int value) {
18	if (!isFull(q)) {
19	q->rear++;
20	if (q->rear == q->data + MAX) {
21	q->rear = q->data;
22	}
23	
24	*q->rear = value;
25	}
26	}
27	
28	void dequeue(Queue* q) {
29	if (!isEmpty(q)) {
30	q->front++;
31	
32	if (q->front == q->data + MAX) {

```

33         q->front = q->data;
34     }
35 }
36 }
37
38 int front(const Queue* q) {
39     if(!isEmpty(q)){
40         return *(q->front);
41     }
42     throw std::runtime_error("Queue kosong");
43 }
44
45 int back(const Queue* q) {
46     if(!isEmpty(q)){
47         return *(q->rear);
48     }
49     throw std::runtime_error("Queue kosong");
50 }

```

B. Pembahasan

Pada baris 3 sampai 6 adalah fungsi `init`. Gunanya untuk inisialisasi queue agar siap dipakai. Cara kerjanya yaitu pointer `front` diarahkan ke alamat data (awal array), sedangkan pointer `rear` diarahkan ke alamat data dikurangi 1 (mundur satu posisi dari awal array), jadi awalnya `front` dan `rear` belum sejajar. Kompleksitas waktu fungsi ini adalah $O(1)$ karena cuma assignment tanpa loop. Kompleksitas ruang fungsi ini adalah $O(1)$ karena tidak ada variabel yang bergantung pada ukuran array.

Pada baris 8 sampai 10 adalah fungsi `isEmpty`. Gunanya untuk mengecek apakah queue kosong atau tidak. Cara kerjanya dengan mengecek apakah `front` menunjuk ke alamat yang sama dengan `rear`? Jika iya, berarti queue kosong, nilainya `true`, kalau tidak, `false`. Kompleksitas waktunya $O(1)$ karna cuma perbandingan dan kompleksitas ruangnya $O(1)$ karna tidak ada variabel tambahan.

Pada baris 12 sampai 14 adalah fungsi `isFull`. Gunanya untuk mengecek apakah queue sudah penuh atau belum. Cara kerjanya dengan mengecek apakah `rear` menunjuk ke alamat data ditambah $(MAX-1)$, yang berarti `rear` sudah di elemen paling belakang dari array. Kalau iya, queue penuh, kalau tidak, belum penuh. Kompleksitas waktunya $O(1)$ karna hanya perbandingan dan kompleksitas ruangnya $O(1)$ karna tidak ada variabel tambahan.

Pada baris 16 sampai 25 adalah fungsi `enqueue`. Gunanya untuk menambahkan nilai baru ke belakang queue. Cara kerjanya pertama mengecek dulu apakah queue sudah penuh atau belum pakai fungsi `isFull`. Kalau belum penuh, `rear` digeser maju satu posisi (`rear++`), lalu dicek lagi apakah `rear` sudah melewati batas akhir array (`data + MAX`)? Kalau iya, `rear` diputar balik ke awal array (circular queue). Baru setelah itu, nilai yang diinput dimasukkan ke posisi `rear` yang baru. Kompleksitas waktunya $O(1)$ karna tidak ada loop dan kompleksitas ruangnya $O(1)$ karna cuma nambah 1 elemen saja.

Pada baris 27 sampai 35 adalah fungsi `dequeue`. Gunanya untuk menghapus nilai paling depan dari queue. Cara kerjanya pertama mengecek dulu apakah queue kosong atau tidak dengan fungsi `isEmpty`. Kalau tidak kosong, `front` digeser maju satu posisi (`front++`), lalu dicek lagi apakah `front` udah ngelewatin batas akhir array? Kalau iya, `front` diputar balik ke awal array. Kompleksitas waktunya $O(1)$ karna tidak ada loop dan kompleksitas ruangnya $O(1)$ karna tidak menambah variabel apapun.

Pada baris 37 sampai 42 adalah fungsi `front`. Gunanya untuk mengakses nilai paling depan queue tanpa menghapusnya. Cara kerjanya pertama mengecek dulu apakah queue kosong atau tidak. Kalau tidak kosong, return nilai yang ditunjuk oleh `front`. Kalau kosong, throw pesan error. Kompleksitas waktunya $O(1)$ karna hanya pengecekan dan return, dan kompleksitas ruangnya $O(1)$ karna tidak ada variabel tambahan.

Pada baris 44 sampai 49 adalah fungsi `back`. Gunanya untuk mengakses nilai paling belakang queue tanpa menghapusnya. Cara kerjanya pertama mengecek dulu apakah queue kosong atau tidak. Kalau tidak kosong, return nilai yang ditunjuk oleh `rear`. Kalau kosong, throw pesan error. Kompleksitas waktunya $O(1)$ karna cuma pengecekan dan return, dan kompleksitas ruangnya $O(1)$ karna tidak memakai variabel tambahan.

SOAL 4

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n : jumlah elemen
- k : ukuran jendela
- A_i : elemen array

Format Input

n k

A_1 A_2 ... A_n

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi.

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 1	3 1 -1 2 6

A. Source Code

Tabel 8. Source Code File prob2.cpp

1	#include "queue.h"
2	#include <iostream>
3	using namespace std;
4	
5	int main() {
6	Queue q;
7	init(&q);
8	int N, k;
9	
10	cin >> N >> k;
11	int A[N];
12	
13	for (int i = 0; i < N; i++) {
14	cin >> A[i];
15	}
16	
17	int hasil = 0;
18	for (int i = 0; i < k; i++){
19	enqueue(&q, A[i]);
20	hasil += A[i];
21	}
22	cout << hasil << " ";
23	
24	for (int i = k; i < N; i++) {
25	int dihapus = front(&q);
26	
27	dequeue(&q);
28	enqueue(&q, A[i]);
29	
30	hasil = hasil - dihapus + A[i];

31	
32	cout << hasil << " ";
33	}
34	return 0;
35	
36	
37	}

B. Output Program

```

PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1> g++ -std=c++23 -Wall -Wextra -o prob2.exe prob2.cpp queue.cpp
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1> .\prob2.exe
5 3
1 2 3 4 5
6 9 12
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1> .\prob2.exe
5 5
1 2 3 4 5
15
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1> .\prob2.exe
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-2-stack-and-queue-soraawistaria-1>

```

Gambar 4. Screenshot Jawaban Soal 4 Modul 2

C. Pembahasan

Pada baris 1 sampai 3 adalah pemanggilan library. Gunanya agar kode di file ini dapat menggunakan fungsi yang tersedia di `queue.h` dan `iostream`. Baris 3 gunanya efisiensi pengetikan.

Pada baris 6 dan 7 adalah deklarasi struct `Queue` dengan nama `q`, lalu dipanggil fungsi `init`. Gunanya untuk mendeklarasi struct agar bisa dipakai sebagai queue dalam program, dan `init` dipanggil untuk menyiapkan pointer `front` dan `rear`.

Baris 8 digunakan untuk deklarasi variabel `N` (total elemen array) dan `k` (ukuran window). Baris 10 digunakan untuk input user, cara kerjanya dengan meminta input nilai dari user, lalu hasil input disimpan ke variabel `N` dan `k`. Baris 11 digunakan untuk deklarasi array `A` sebanyak `N` elemen.

Pada baris 13 sampai 15 adalah looping, digunakan agar mendapatkan input user secara berulang untuk mengisi array `A`. Cara kerjanya yaitu program iterasi sebanyak `N` kali, dan setiap iterasi user input nilai yang disimpan ke `A[i]`.

Baris 17 digunakan untuk inisialisasi variabel `hasil` (penampung total nilai) dengan nilai 0.

Pada baris 18 sampai 21 adalah looping, digunakan untuk mengisi queue pertama kali sebanyak k elemen sekaligus menghitung total nilainya. Cara kerjanya yaitu program iterasi sebanyak k kali, setiap iterasi nilai $A[i]$ di-enqueue ke queue, lalu nilai itu juga ditambahkan ke variabel `hasil`.

Baris 22 digunakan untuk mencetak nilai hasil.

Pada baris 24 sampai 33 adalah looping utama, digunakan untuk menggeser window dari indeks k sampai $N-1$ (sliding window). Cara kerjanya yaitu program iterasi dari $i = k$ sampai sebelum N . Tiap iterasi, nilai paling depan queue diambil dulu ke variabel dihapus (dengan fungsi `front`), lalu nilai itu di-dequeue. Setelah itu nilai baru $A[i]$ di-enqueue ke queue (masuk ke belakang). Baris 30 digunakan untuk update nilai hasil, caranya hasil dikurangi nilai yang dihapus lalu ditambah nilai yang baru masuk ($A[i]$). Baris 32 digunakan untuk menampilkan nilai hasil window yang sudah diupdate itu.

Kompleksitas ruang program ini adalah $O(n)$ karena ada array A dan queue yang menyimpan elemen sebanyak ukuran window k . Kompleksitas waktunya $O(n)$ karena cuma ada loop-loop yang jalan linear sebanyak N kali, dan di dalamnya fungsi enqueue/dequeue/front semuanya $O(1)$ sehingga tidak nambah kompleksitas.

MODUL 3:

LINKED LIST

SOAL 1

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 9. Source Code File `single_linked_list.cpp`

1	<code>#include "single_linked_list.h"</code>
2	<code>#include <iostream></code>
3	<code>using namespace std;</code>
4	
5	<code>void SingleLinkedList::init() {</code>
6	<code> clear();</code>
7	<code> head = nullptr;</code>
8	<code> tail = nullptr;</code>
9	<code>}</code>
10	
11	<code>bool SingleLinkedList::is_empty() {</code>
12	<code> if (head == nullptr && tail == nullptr) {</code>
13	<code> return true;</code>
14	<code> }</code>
15	<code> return false;</code>
16	<code>}</code>
17	
18	<code>void SingleLinkedList::add_front(int val) {</code>
19	<code> // nambah di depan</code>
20	<code> Node * newNode = new Node;</code>
21	<code> if (is_empty()) {</code>
22	<code> head = tail = newNode;</code>
23	<code> newNode->data = val;</code>
24	<code> tail->next = head;</code>

```

25     } else {
26         newNode->data = val;
27         newNode->next = head;
28         head = newNode;
29         tail->next = head;
30     }
31     size++;
32 }
33
34 void SingleLinkedList::add_back(int val) {
35     Node * newNode = new Node;
36     newNode->data = val;
37     newNode->next = head;
38     if (is_empty()) {
39         add_front(val);
40         return;
41     } else {
42         tail->next = newNode;
43         tail = newNode;
44     }
45     size++;
46 }
47
48 void SingleLinkedList::add_idx(int val, int idx) {
49     if (idx==0) {
50         add_front(val);
51         return;
52     } else if (idx < 0) {
53         throw std::runtime_error("Woi input indeks yang
bener");
54     } else if (idx > size) {
55         throw std::runtime_error("KELEBIHAN
INDEKSNYA!");
56     } else if (idx == size) {
57         add_back(val);
58         return;
59     } else {
60         Node * newNode = new Node;
61         newNode->data = val;
62         Node * curr = head;
63         for (int i=0; i<idx-1; i++) {
64             curr = curr->next;
65         }
66         newNode->next = curr->next;
67         curr->next = newNode;
68     }
69     size++;

```

```

70     }
71
72     void SingleLinkedList::delete_front() {
73         if (is_empty()) return;
74         if (head == tail) {
75             delete head;
76             head = nullptr;
77             tail = nullptr;
78             size = 0;
79         } else {
80             Node * curr = head;
81             head = head->next;
82             tail->next = head;
83
84             delete curr;
85             size--;
86         }
87     }
88
89     void SingleLinkedList::delete_back() {
90         if (is_empty()) return;
91
92         if (head == tail) { //jika hanya 1 node
93             delete head;
94             head = tail = nullptr;
95             size--;
96         } else {
97             Node * curr = head;
98             while (curr->next != tail) {
99                 curr = curr->next;
100             }
101             delete tail;
102             tail = curr;
103             tail->next = head;
104             size--;
105         }
106     }
107
108     void SingleLinkedList::delete_idx(int idx) {
109         if (is_empty()) return;
110         if (idx==0) {
111             delete_front();
112             return;
113         }
114         Node * curr = head;
115         for (int i=0; i<idx-1; i++) {
116             curr = curr->next;

```

```

117     }
118     Node * temp = curr->next;
119     curr->next = temp->next;
120
121     if (temp == tail) tail = curr; //memindah tail
122
123     delete temp;
124     size--;
125 }
126
127 void SingleLinkedList::display() {
128     if (!is_empty()) {
129         Node * curr;
130         curr = head;
131         do {
132             cout << curr->data << " ";
133             curr = curr->next;
134         } while (curr != head);
135
136         cout << endl;
137     } else {
138         cout << "List Kosong" << endl;
139     }
140 }
141
142 int SingleLinkedList::get(int idx) {
143     if (is_empty()) throw std::runtime_error("List
kosong");
144
145     Node * curr = head;
146     for (int i=0; i<idx; i++) {
147         curr = curr->next;
148     }
149     return curr->data;
150 }
151
152 void SingleLinkedList::set(int val, int idx) {
153     if (is_empty()) return;
154
155     Node * curr = head;
156     for (int i=0; i<idx; i++) {
157         curr = curr->next;
158     }
159     curr->data = val;
160 }
161
162 void SingleLinkedList::clear() {

```

```

163     while (!is_empty()) {
164         Node * curr = head;
165         if (size == 1) { head = tail = nullptr; }
166         else {
167             head = head->next;
168             tail->next = head;
169         }
170         delete curr;
171         size--;
172     }
173     head = nullptr;
174     tail = nullptr;
175     size = 0;
176 }

```

B. Pembahasan

Pada baris 1 dan 2 adalah pemanggilan library. Gunanya agar kode di file ini dapat menggunakan fungsi yang tersedia di `single_linked_list.h` dan `iostream`. Baris 3 gunanya efisiensi pengetikan.

Pada baris 5 sampai 9 adalah fungsi inisialisasi. Gunanya untuk inisialisasi pointer `head` dan `tail`. Cara kerjanya yaitu menggunakan fungsi `clear` dulu agar benar benar tidak ada linked list sebelumnya, lalu inisialisasi pointer `head` dan `tail` di `nullptr` (pointer kosong) karena belum ada linked list. Kompleksitas waktu fungsi ini adalah $O(1)$ karena fungsi berjalan statis tanpa dipengaruhi nilai variabel lain. Kompleksitas ruang fungsi ini adalah $O(1)$ karena tidak ada variabel array yang bergantung pada nilai ukuran array.

Pada baris 11 sampai 16 adalah fungsi `is_empty`. Gunanya untuk mengecek apakah linked list kosong atau tidak. Cara kerjanya dengan mengecek apakah `head` dan `tail` menunjuk ke `nullptr`? Jika iya, nilainya `true`, jika tidak, nilainya `false`.

Pada baris 18 sampai 32 adalah fungsi `add_front`. Gunanya untuk menambahkan node di depan linked list. Cara kerjanya dimulai dari menambah node baru di variabel `newNode`. Lalu mengecek apakah linked list kosong atau tidak. Kalau kosong, node baru akan ditunjuk oleh `head` dan `tail`, lalu menambahkan nilai yang diinput ke data node baru. Agar menjadi circular, pointer `next` si node baru menunjuk ke `head`. Kalau linked list tidak kosong, node baru diisi nilai, lalu mengatur pointer `next` si node baru ke `head`, lalu pointer `head` dipindah ke node baru, dan `next` nya `tail` diarahkan ke `head` yang baru (node baru tadi).

Kompleksitas ruangnya $O(1)$ karna variabelnya konstan dan kompleksitas waktunya $O(1)$ karna program berjalan konstan tanpa ada loop.

Pada baris 34 sampai 46 adalah fungsi `add_back`. Gunanya untuk menambahkan node di belakang (node terakhir) linked list. Cara kerjanya dengan membuat pointer `newNode` dulu, lalu menambahkan nilai ke data node baru, lalu mengatur pointer `next` nya node baru untuk menunjuk ke head. Kalau linked list kosong, jalankan fungsi `add_front`. Kalau tidak kosong, atur `tail` `next` untuk menunjuk ke node baru dan barulah pointer `tail` dipindah ke node baru. Kompleksitas ruangnya $O(1)$ karna menambah 1 node aja dan kompleksitas waktunya $O(1)$ karna tidak ada looping. Tapi kalau fungsi ini terus dipanggil, kompleksitas ruangnya jadi $O(n)$ karna linked list emang linear kompleksitasnya.

Pada baris 48 sampai 70 adalah fungsi `add_idx`. Gunanya untuk menambahkan node di indeks tertentu, disisipkan diantara node lain. Cara kerjanya pertama ngecek apakah indeksnya 0? Kalau iya, panggil fungsi `add_front`. Lalu apakah indeks kurang dari 0? Maka throw pesan error. Lalu apakah indeks lebih dari size linkedlist? Kalau iya maka throw pesan error out of index. Lalu apakah nilai indeks sama dengan nilai size? Kalau iya, berarti ini nilai terakhir linked list, jadi jalankan fungsi `add_back` saja. Lalu kalau memang indeks ada pada sesudah head dan sebelum tail, maka buat node baru dan masukkan nilai ke node baru. Lalu buat pointer `curr` menunjuk ke head, di traverse dengan loop hingga indeks-1. Lalu atur pointer `next` si node baru menunjuk ke node sesudah `curr`, lalu `next` nya `curr` menunjuk ke node baru tadi. Sehingga node baru berada diantara `curr` dan `curr->next` itu. Kompleksitas ruangnya adalah $O(n)$ karna ini membuat linked list dan kompleksitas waktunya $O(n)$ karna melakukan traverse dengan loop.

Pada baris 72 sampai 87 adalah fungsi `delete_front`. Gunanya agar bisa menghapus node paling depan si linked list. Cara kerjanya pertama tu fungsi mengecek apakah linked list kosong atau tidak (kalau kosong, kembali). Lalu mengecek apakah nilai head sama dengan tail (apakah cuma ada 1 node)? Kalau iya, hapus node head dan jadikan head dan tail menunjuk ke `nullptr` serta ubah size menjadi 0. Kalau node tidak satu-satunya di linked list, buat node `curr` yang menunjuk ke head lalu geser pointer head ke sesudah si head ini lalu ubah `next` nya si tail agar menunjuk ke head baru. Setelah memindah-mindah pointer, baru node `curr` yang menunjuk di depan tadi dihapus dan kurangi size. Kompleksitas

ruangnya $O(1)$ karna ruang yang digunakan ketika fungsi ini berjalan itu konstan dan kompleksitas waktu juga $O(1)$ karna tidak ada looping.

Pada baris 89 sampai 106 adalah fungsi `delete_black`. Gunanya agar bisa menghapus node paling belakang. Cara kerjanya pertama itu ngecek apakah linked list kosong atau tidak (kalau iya, kembalikan/gausah jalan fungsinya). Lalu mengecek apakah Cuma ada 1 node? Kalau iya, hapus head, ubah head dan tail menunjuk nullptr, kurangi size. Jika node lebih dari 1 tadi, masuk ke else. Buat node curr yang menunjuk ke head, lalu traverse sampai sebelum tail, lalu hapus tail, lalu geser tail ke curr, lalu next nya si tail diubah menunjuk ke head. Lalu kurangi size. Kompleksitas ruangnya $O(n)$ karna ada traverse, menggunakan ruang sebanyak linked list itu dan kompleksitas waktunya $O(n)$ karna ada traverse dengan loop.

Pada baris 108 sampai 125 adalah fungsi `delete_idx`. Gunanya agar bisa menghapus node tertentu yang ada di tengah linkedlist (setelah head, sebelum tail). Cara kerjanya pertama mengecek apakah linked list kosong atau tidak (kalau kosong ya kembali). Lalu kalau idxnya 0, jalankan `delete_front`. Kalau tidak keduanya, fungsinya jalan mulai dari membuat node curr yang akan di traverse. Lalu node baru *temp yang menunjuk sesudah node curr. Sebelum menghapus *temp, nextnya curr dipindah ke node sesudah temp. Misalnya kebetulan temp nya ini ternyata tail, maka tailnya digeser ke curr dulu. Lanjut menghapus temp dan mengurangi size. Kompleksitas ruangnya $O(n)$ karna ada traverse, menggunakan ruang sebanyak linked list itu dan kompleksitas waktunya $O(n)$ karna ada traverse dengan loop.

Pada baris 127 sampai 140 adalah fungsi `display`. Gunanya agar bisa menampilkan seluruh nilai linked list. Berjalan kalau linked list tidak kosong, kalau kosong cetak "list kosong". Nah kalau tidak kosong, buat node curr untuk traverse. Selama traverse, cetak juga isi data si curr. Kompleksitas ruangnya adalah $O(1)$ karna variabel curr konstan menyimpan 1 nilai dan kompleksitas waktunya adalah $O(n)$ karna brute force untuk mencetak nilai data setiap node.

Pada baris 142 sampai 150 adalah fungsi `get`. Gunanya agar bisa mengambil nilai yang ada dalam data node. Cara kerjanya, pertama mengecek apakah linked list kosong atau tidak, kalau kosong throw pesan error. Kalau tidak, buat node curr untuk traverse sampai indeks yang diminta. Kalau sudah, kembalikan nilai data dalam node curr. Kompleksitas

ruangnya adalah $O(1)$ karna variabel `curr` konstan menyimpan 1 nilai dan kompleksitas waktunya adalah $O(n)$ karna traverse mencari node `idx`.

Pada baris 152 sampai 160 adalah fungsi `set`. Gunanya agar bisa mengganti nilai dalam data node tertentu. Cara kerjanya pertama mengecek linked list kosong atau tidak, kalau kosong return. Kalau tidak, buat node `curr` dan traverse. Sesudah traverse, ganti nilai data di `curr` dengan nilai baru. Kompleksitas ruanganya adalah $O(1)$ karna variabel `curr` konstan dan kompleksitas waktunya adalah $O(n)$ karna traverse hingga node `idx`.

Pada baris 162 sampai 176 adalah fungsi `clear`. Gunanya untuk membersihkan atau menghapus semua node linked list. Selama linked list tidak kosong, traverse berjalan dengan membuat node `curr` setiap iterasi namun juga menghapus `curr` tersebut setiap iterasi. Kalau cuma ada 1 node, langsung `nullptr` kan saja `head` dan `tailnya`. Terakhir, ubah `head` dan `tail` menunjuk ke `nullptr` dan ubah `size` menjadi 0. Kompleksitas ruanganya adalah $O(1)$ karna variabel `curr` konstan dan kompleksitas waktunya adalah $O(n)$ karna brute force untuk menghapus setiap node.

SOAL 2

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh N buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh K langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ($K = K + 1$).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ($K = K - 1$).
- Jika pada suatu titik energi lompatan habis atau negatif ($K \leq 0$), sistem akan mengalami reset dan mengembalikan nilai lompatan ke nilai K awal (saat pancaran energi pertama kali diinisialisasi).

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

Batasan

- $1 \leq N \leq 10^4$
- $1 \leq K \leq 10^9$
- $1 \leq A_i \leq 10^6$ (Nilai ukiran pada batu)
- Program wajib menggunakan implementasi struktur data yang dibuat dari Soal 1

Format Input

N K

A_1 A_2 ... A_N

Format Output

Sebuah bilangan bulat tunggal yang merupakan nilai ukiran pada batu terakhir yang tersisa.

Contoh Kasus

Input	Output
5 2 1 4 3 6 2	6
4 5 10 11 12 13	12

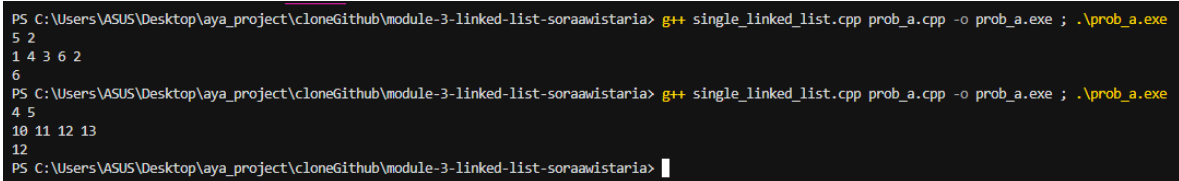
A. Source Code

Tabel 10. Source Code File prob_a.cpp

```

1  #include "single_linked_list.h"
2  #include <iostream>
3  using namespace std;
4
5  int main () {
6      SingleLinkedList sll;
7      sll.init();
8
9      int N, K, x;
10     cin >> N >> K;
11
12     for (int i = 0; i < N; i++) {
13         cin >> x;
14         sll.add_back(x);
15     }
16     int k_awal = K;
17     int idx = 0;
18     while (sll.size > 1) {
19         idx = (idx + K - 1) % sll.size;
20
21         if (sll.get(idx) % 2 == 0) {
22             K++;
23         } else {
24             K--;
25         }
26         sll.delete_idx(idx);
27
28         if (K <= 0) K = k_awal;
29
30         idx = idx % sll.size;
31     }
32     sll.display();
33 }
```

B. Output Program



```
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-3-linked-list-soraawistaria> g++ single_linked_list.cpp prob_a.cpp -o prob_a.exe ; .\prob_a.exe
5 2
1 4 3 6 2
6
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-3-linked-list-soraawistaria> g++ single_linked_list.cpp prob_a.cpp -o prob_a.exe ; .\prob_a.exe
4 5
10 11 12 13
12
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-3-linked-list-soraawistaria>
```

Gambar 5. Screenshot Jawaban Soal 2 Modul 3

C. Pembahasan

Pada baris 1 dan 2 adalah pemanggilan library. Gunanya agar kode di file ini dapat menggunakan fungsi yang tersedia di `single_linked_list.h` dan `iostream`. Baris 3 gunanya efisiensi pengetikan.

Pada baris 6 adalah deklarasi struct single linked list dengan nama `sll`. Gunanya untuk mendeklarasi struct agar variabel dan fungsi di dalam struct bisa digunakan untuk kode program soal 2. Baris 7 adalah pemanggilan fungsi `init`, karna menginisialisasi pointer `head` dan `tail` sebelum dipakai dalam program.

Pada baris 9 digunakan untuk deklarasi variabel yang akan digunakan dalam program. Baris 10 digunakan untuk input user, cara kerjanya dengan meminta input nilai dari user, lalu hasil input user disimpan ke variabel `N` (total nilai) dan `K` (loncatan batu).

Pada baris 12 sampai 15 adalah looping, digunakan agar mendapatkan input user secara berulang dan juga digunakan untuk menambahkan data input user ke linked list. Cara kerjanya yaitu program akan iterasi sebanyak `N` kali, dan sebanyak `N` kali ini user harus menginputkan nilai. Nilai input masuk ke variabel `x`. Baris 14 digunakan untuk memanggil fungsi `add_back`, cara kerjanya fungsi harus diisi argumen (variabel `x`) agar nilai variabel `x` masuk ke linked list.

Pada baris 16 dan 17 adalah inisialisasi variabel. Baris 16 digunakan untuk menyimpan nilai `K` untuk dipakai pada program, baris 17 digunakan untuk inisialisasi indeks 0 agar bisa menghitung indeks traverse.

Pada baris 18 sampai 31 adalah perulangan selama batu reliki masih lebih dari 1. Digunakan untuk traverse berulang-ulang hingga batu reliki tersisa 1 dan untuk menghapus node yang menjadi target. Cara kerjanya yaitu menginisialisasi indeks node yang akan dihapus. Lalu mengecek apakah data pada node tersebut nilainya genap atau ganjil. Kalau

genap, nilai K ditambah 1, kalau ganjil nilai K dikurangi 1. Setelah itu node tersebut dihapus (batu dihancurkan).

Pada baris 28 digunakan untuk menghindari nilai K menjadi minus dengan mengembalikan nilai K ke nilai K awal jika minus. Baris 30 digunakan untuk menggunakan lagi nilai idx. Baris 32 digunakan untuk menampilkan nilai node yang tersisa. Kompleksitas ruang program ini adalah $O(n)$ karena ada linked list dan kompleksitas waktunya $O(n^2)$ karena ada while yang mengulang-ulang iterasi hingga node tersisa 1 dan di dalam while ada fungsi `delete_idx` yang juga menggunakan while loop ketika berjalan.

SOAL 3

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList` beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 11. Source Code File `double_linked_list.cpp`

1	<code>#include "double_linked_list.h"</code>
2	<code>#include <iostream></code>
3	<code>using namespace std;</code>
4	
5	<code>void DoubleLinkedList::init() {</code>
6	<code> clear();</code>
7	<code> head = nullptr;</code>
8	<code> tail = nullptr;</code>
9	<code>}</code>
10	
11	<code>bool DoubleLinkedList::is_empty() {</code>
12	<code> if (head == nullptr && tail == nullptr) {</code>
13	<code> return true;</code>
14	<code> }</code>
15	<code> return false;</code>
16	<code>}</code>
17	
18	<code>void DoubleLinkedList::add_front(char val) {</code>
19	<code> //tambah deoan</code>
20	<code> Node * newNode = new Node;</code>
21	<code> newNode->data = val;</code>
22	<code> if (is_empty()) {</code>
23	<code> head = tail = newNode;</code>
24	<code> tail->next = head;</code>
25	<code> tail->prev = newNode;</code>
26	<code> } else {</code>
27	<code> newNode->next = head;</code>
28	<code> head = newNode;</code>
29	

```

30         tail->next = head;
31         newNode->prev = tail;
32     }
33     size++;
34 }
35
36 void DoubleLinkedList::add_back(char val) {
37     if (is_empty()) {
38         add_front(val);
39         return;
40     }
41     Node * newNode = new Node;
42     newNode->data = val;
43     newNode->next = head;
44     newNode->prev = tail;
45
46     tail->next = newNode;
47     tail = newNode;
48     head->prev = tail;
49     size++;
50 }
51
52 void DoubleLinkedList::add_idx(char val, int idx) {
53     //insert
54     if (idx == 0) {
55         add_front(val);
56         return;
57     } else if (idx < 0) {
58         throw std::runtime_error("Woi input indeks yang
bener");
59     } else if (idx > size) {
60         throw std::runtime_error("KELEBIHAN
INDEKSNYA!");
61     } else if (idx == size) {
62         add_back(val);
63         return;
64     } else {
65         Node * newNode = new Node;
66         newNode->data = val;
67         Node * curr = head;
68         for (int i=0; i < idx-1; i++) {
69             curr = curr->next;
70         }
71         newNode->next = curr->next;
72         newNode->prev = curr;
73         curr->next = newNode;
74         curr->next->prev = newNode;

```

```

75     }
76     size++;
77 }
78
79 void DoubleLinkedList::delete_front() {
80     //hapus depan
81     if (is_empty()) return;
82     if (head == tail) {
83         delete head;
84         head = nullptr;
85         tail = nullptr;
86         size = 0;
87     } else {
88         Node * curr = head;
89         head = head->next;
90         head->prev = tail;
91         tail->next = head;
92
93         delete curr;
94         size--;
95     }
96 }
97
98 void DoubleLinkedList::delete_back() {
99     if (is_empty()) return;
100    if (head == tail) {
101        delete_front();
102        return;
103    } else {
104        Node * curr = head;
105        while (curr->next != tail) {
106            curr = curr->next;
107        }
108        delete tail;
109        tail = curr;
110        tail->next = head;
111        head->prev = tail;
112        size--;
113    }
114 }
115
116 void DoubleLinkedList::delete_idx(int idx) {
117     if (is_empty()) return;
118     Node * nodePtr = head;
119     int count = 0;
120
121     if (idx==0) {

```

```

122         delete_front();
123         return;
124     }
125     if (nodePtr->next == head) {
126         delete_back ();
127         return;
128     }
129     while (count < idx-1 && nodePtr->next != head) {
130         nodePtr = nodePtr->next;
131         count++;
132     }
133     Node * target = nodePtr->next;
134     nodePtr->next = target->next;
135     target->next->prev = nodePtr;
136
137     delete target;
138     size--;
139 }
140
141 void DoubleLinkedList::display() {
142     if (!is_empty()) {
143         Node * curr;
144         curr = head;
145         do {
146             cout << curr->data;
147             curr = curr->next;
148         } while (curr != head);
149
150         cout << endl;
151     } else {
152         cout << "List Kosong" << endl;
153     }
154 }
155
156 char DoubleLinkedList::get(int idx) {
157     if (is_empty()) throw std::runtime_error("List
kosong");
158
159     Node * curr = head;
160     for (int i=0; i<idx; i++) {
161         curr = curr->next;
162     }
163     return curr->data;
164 }
165
166 void DoubleLinkedList::set(char val, int idx) {
167     if (is_empty()) return;

```



```

168
169     Node * curr = head;
170     for (int i=0; i<idx; i++) {
171         curr = curr->next;
172     }
173     curr->data = val;
174 }
175
176 void DoubleLinkedList::clear() {
177     while (!is_empty()) {
178         Node * curr = head;
179         if (size == 1) { head = tail = nullptr; }
180         else {
181             head = head->next;
182             tail->next = head;
183         }
184         delete curr;
185         size--;
186     }
187     head = nullptr;
188     tail = nullptr;
189     size = 0;
190 }

```

B. Pembahasan

Pada baris 1 dan 2 adalah pemanggilan library. Gunanya agar kode di file ini dapat menggunakan fungsi yang tersedia di `single_linked_list.h` dan `iostream`. Baris 3 gunanya efisiensi pengetikan.

Pada baris 5 sampai 9 adalah fungsi inisialisasi (`init`). Gunanya untuk inisialisasi pointer head dan tail. Cara kerjanya yaitu menggunakan fungsi `clear` dulu agar benar benar tidak ada linked list sebelumnya, lalu inisialisasi pointer head dan tail di `nullptr` (pointer kosong) karena belum ada linked list. Kompleksitas waktu fungsi ini adalah $O(1)$ karena fungsi berjalan statis tanpa dipengaruhi nilai variabel lain. Kompleksitas ruang fungsi ini adalah $O(1)$ karena tidak ada variabel array yang bergantung pada nilai ukuran array.

Pada baris 11 sampai 16 adalah fungsi `is_empty`. Gunanya untuk mengecek apakah linked list kosong atau tidak. Cara kerjanya dengan mengecek apakah head dan tail menunjuk ke `nullptr`? Jika iya, nilainya `true`, jika tidak, nilainya `false`.

Pada baris 18 sampai 34 adalah fungsi `add_front`. Gunanya untuk menambahkan node di depan linked list. Cara kerjanya dimulai dari menambah node baru di variabel

newNode dan memasukkan nilai val ke data newNode. Lalu mengecek apakah linked list kosong atau tidak. Kalau kosong, node baru akan ditunjuk oleh head dan tail. Agar menjadi circular, pointer next si tail baru menunjuk ke head dan pointer prev si tail menunjuk ke newNode. Kalau linked list tidak kosong, mengatur pointer next si node baru menunjuk ke head, lalu pointer head dipindah ke node baru, dan next nya tail diarahkan ke head yang baru (node baru tadi), serta mengisi nilai prev si node baru untuk menunjuk ke tail agar circular. Kompleksitas ruangnya $O(1)$ karna variabelnya konstan dan kompleksitas waktunya $O(1)$ karna program berjalan konstan tanpa ada loop.

Pada baris 36 sampai 50 adalah fungsi `add_back`. Gunanya untuk menambahkan node di belakang (node terakhir) linked list. Cara kerjanya dengan mengecek apakah linked list kosong atau tidak (kalau kosong, gunakan fungsi `add_front`). Lalu membuat pointer newNode dulu, lalu menambahkan nilai ke data node baru, lalu menggeser pointer next nya node baru untuk menunjuk ke head dan mengubah prevnya ke menunjuk tail. Kemudian geser tail next untuk menunjuk ke node baru dan barulah pointer tail dipindah ke node baru dan prev dari head diubah menunjuk ke tail yang sekarang. Kemudian size ditambah. Kompleksitas ruangnya $O(1)$ karna menambah 1 node aja dan kompleksitas waktunya $O(1)$ karna tidak ada looping.

Pada baris 52 sampai 77 adalah fungsi `add_idx`. Gunanya untuk menambahkan node di indeks tertentu, disisipkan diantara node lain. Cara kerjanya pertama mengecek apakah indeksnya 0? Kalau iya, panggil fungsi `add_front`. Lalu apakah indeks kurang dari 0? Maka throw pesan error. Lalu apakah indeks lebih dari size linkedlist? Kalau iya maka throw pesan error out of index. Lalu apakah nilai indeks sama dengan nilai size? Kalau iya, berarti ini nilai terakhir linked list, jadi jalankan fungsi `add_back` saja. Lalu kalau memang indeks ada pada sesudah head dan sebelum tail, maka buat node baru dan masukkan nilai ke node baru. Lalu buat pointer curr menunjuk ke head, di traverse dengan loop hingga indeks-1. Lalu atur pointer next si node baru menunjuk ke node sesudah curr dan prev nya node baru menunjuk ke curr. Lalu next nya curr menunjuk ke node baru tadi. Sehingga node baru berada diantara curr dan curr->next itu, serta prevnya si curr->next diarahkan ke node newNode. Kompleksitas ruangnya adalah $O(n)$ karna ini membuat linked list dan kompleksitas waktunya $O(n)$ karna melakukan traverse dengan loop.

Pada baris 79 sampai 96 adalah fungsi `delete_front`. Gunanya agar bisa menghapus node paling depan si linked list. Cara kerjanya pertama tu fungsi mengecek apakah linked list kosong atau tidak (kalau kosong, kembali). Lalu mengecek apakah nilai head sama dengan tail (apakah cuma ada 1 node)? Kalau iya, hapus node head dan jadikan head dan tail menunjuk ke nullptr serta ubah size menjadi 0. Kalau node tidak satu-satunya di linked list, buat node curr yang menunjuk ke head lalu geser pointer head ke sesudah si head ini lalu ubah next nya si tail agar menunjuk ke head baru. Tambahan, prev nya head diarahkan ke tail. Setelah memindah-mindah pointer, baru node curr yang menunjuk di depan tadi dihapus dan kurangi size. Kompleksitas ruangnya $O(1)$ karna ruang yang digunakan ketika fungsi ini berjalan itu konstan dan kompleksitas waktu juga $O(1)$ karna tidak ada looping.

Pada baris 98 sampai 114 adalah fungsi `delete_back`. Gunanya agar bisa menghapus node paling belakang. Cara kerjanya pertama itu ngecek apakah linked list kosong atau tidak (kalau iya, kembalikan/gausah jalan fungsinya). Lalu mengecek apakah Cuma ada 1 node? Kalau iya, panggil fungsi `delete_front`. Jika node lebih dari 1 tadi, masuk ke else. Buat node curr yang menunjuk ke head, lalu traverse sampai sebelum tail, lalu hapus tail, lalu geser tail ke curr, lalu next nya si tail diubah menunjuk ke head dan prev nya head menunjuk ke tail. Lalu kurangi size. Kompleksitas ruangnya $O(n)$ karna ada traverse, menggunakan ruang sebanyak linked list itu dan kompleksitas waktunya $O(n)$ karna ada traverse dengan loop.

Pada baris 116 sampai 139 adalah fungsi `delete_idx`. Gunanya agar bisa menghapus node tertentu yang ada di tengah linkedlist (setelah head, sebelum tail). Cara kerjanya pertama mengecek apakah linked list kosong atau tidak (kalau kosong ya kembali). Lalu kalau idxnya 0, jalankan `delete_front`. Kalau tidak keduanya, fungsinya jalan mulai dari membuat node nodePtr yang akan di traverse dan variabel count untuk parameter while. Lalu traverse nodePtr dengan while sampai indeks-1 dan tidak sampai tail. Setelah traverse, buat node target yang menunjuk ke node sesudah nodePtr, lalu next nya nodePtr diarahkan ke node sesudah target, lalu prev nya si node sesudah node target tadi diarahkan ke nodePtr (jadi, target ada di antara nodePtr dan target->next). Setelah itu, node target dihapus dan size dikurangi. Kompleksitas ruangnya $O(n)$ karna ada traverse, menggunakan ruang sebanyak linked list itu dan kompleksitas waktunya $O(n)$ karna ada traverse dengan loop.

Pada baris 141 sampai 154 adalah fungsi `display`. Gunanya agar bisa menampilkan seluruh nilai linked list. Berjalan kalau linked list tidak kosong, kalau kosong cetak "list kosong". Nah kalau tidak kosong, buat node `curr` untuk traverse. Selama traverse, cetak juga isi data si `curr`. Kompleksitas ruangnya adalah $O(1)$ karna variabel `curr` konstan menyimpan 1 nilai dan kompleksitas waktunya adalah $O(n)$ karna brute force untuk mencetak nilai data setiap node.

Pada baris 156 sampai 164 adalah fungsi `get`. Gunanya agar bisa mengambil nilai yang ada dalam data node. Cara kerjanya, pertama mengecek apakah linked list kosong atau tidak, kalau kosong throw pesan error. Kalau tidak, buat node `curr` untuk traverse sampai indeks yang diminta. Kalau sudah, kembalikan nilai data dalam node `curr`. Kompleksitas ruangnya adalah $O(1)$ karna variabel `curr` konstan menyimpan 1 nilai dan kompleksitas waktunya adalah $O(n)$ karna traverse mencari node `idx`.

Pada baris 166 sampai 174 adalah fungsi `set`. Gunanya agar bisa mengganti nilai dalam data node tertentu. Cara kerjanya pertama mengecek linked list kosong atau tidak, kalau kosong return. Kalau tidak, buat node `curr` dan traverse. Sesudah traverse, ganti nilai data di `curr` dengan nilai baru. Kompleksitas ruangnya adalah $O(1)$ karna variabel `curr` konstan dan kompleksitas waktunya adalah $O(n)$ karna traverse hingga node `idx`.

Pada baris 176 sampai 190 adalah fungsi `clear`. Gunanya untuk membersihkan atau menghapus semua node linked list. Selama linked list tidak kosong, traverse berjalan dengan membuat node `curr` setiap iterasi namun juga menghapus `curr` tersebut setiap iterasi. Kalau cuma ada 1 node, langsung `nullptr` kan saja `head` dan `tail`-nya. Terakhir, ubah `head` dan `tail` menunjuk ke `nullptr` dan ubah `size` menjadi 0. Kompleksitas ruangnya adalah $O(1)$ karna variabel `curr` konstan dan kompleksitas waktunya adalah $O(n)$ karna brute force untuk menghapus setiap node.

SOAL 4

Si Palui mengamati fenomena aneh di atas sebuah gelanggang melingkar yang berisi N karakter. Terdapat dua proyektor anomali, Alpha (dimulai dari head, bergerak maju menggunakan next) dan Beta (dimulai dari tail, bergerak mundur menggunakan prev).

Palui melakukan observasi selama R ronde. Pada setiap ronde:

1. Proyektor Alpha dipaksa melompat maju sebanyak X langkah. Proyektor Beta dipaksa melompat mundur sebanyak Y langkah (simultan).
2. Tabrakan Singular: Jika Alpha dan Beta mendarat di titik atau karakter yang sama persis secara bersamaan, karakter tersebut akan hancur dan lenyap dari lingkaran dimensi. Jika ini terjadi, Alpha berpindah ke karakter next dari karakter yang hancur, dan Beta berpindah ke karakter prev dari karakter yang hancur.
3. Resonansi Bersebelahan (Adjacent): Jika Alpha dan Beta saling bersebelahan langsung, sifat kuantum dari kedua karakter tersebut saling bertukar (data antara kedua node tersebut di-swap).

Tentukan sisa karakter pada lingkaran dimensi setelah R ronde selesai dieksekusi.

Batasan

- $1 \leq N \leq 5000$
- $1 \leq R \leq 5000$
- $1 \leq X, Y \leq 10^{18}$
- Struktur wajib menggunakan implementasi struktur data dari Soal 3.

Keterangan:

- n : jumlah elemen
- k : ukuran jendela
- A_i : elemen array

Format Input

N R

C_1 C_2 ... C_N

X_1 Y_1

X2 Y2

...

XR YR

Keterangan: C_i adalah satu karakter char tunggal.

Format Output

Karakter yang tersisa, dicetak berurutan mulai dari head hingga ke elemen terakhir, tanpa spasi. Jika dimensi menjadi kosong, cetak EMPTY.

Contoh Kasus

Input	Output
4 2 A B C D 1 1 2 3	ACB
5 3 V W X Y Z 100000 100000 2 2 5 5	VWYZ

A. Source Code

Tabel 12. Source Code File prob_b.cpp

```
1  #include "double_linked_list.h"
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      DoubleLinkedList dll;
7      int N, R;
8      char C;
9      dll.init();
10
11      cin >> N >> R;
```

```

12     for (int i=0; i < N; i++) {
13         cin >> C;
14         dll.add_back(C);
15     }
16     int X[R], Y[R];
17     for (int i=0; i<R; i++) {
18         cin >> X[i] >> Y[i];
19     }
20
21     cout << endl;
22     Node * alpha = dll.head;
23     Node * beta = dll.tail;
24     for (int i=0; i<R; i++) {
25
26         for (int j=0; j < X[i] ; j++) {
27             alpha = alpha->next;
28         }
29         for (int k=0; k < Y[i] ;k++) {
30             beta = beta->prev;
31         }
32
33         if (alpha->next == beta || alpha->prev == beta)
34     {
35         char swap_beta = beta->data;
36         beta->data = alpha->data;
37         alpha->data = swap_beta;
38     } else if (alpha == beta) {
39         Node * target = dll.head;
40         int idx = 0;
41
42         while (target != alpha) {
43             target = target->next;
44             idx++;
45         }
46         alpha = target->next;
47         beta = target->prev;
48         dll.delete_idx(idx);
49     }
50     dll.display();
51 }

```

B. Output Program

```
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-3-linked-list-soraawistaria> g++ double_linked_list.cpp prob_b.cpp -o prob_b.exe ; .\prob_b.exe
4 2
A B C D
1 1
2 3

ACB
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-3-linked-list-soraawistaria> g++ double_linked_list.cpp prob_b.cpp -o prob_b.exe ; .\prob_b.exe
5 3
V W X Y Z
100000 100000
2 2
5 5

VWYZ
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-3-linked-list-soraawistaria>
```

Gambar 6. Screenshot Jawaban Soal 4 Modul 3

C. Pembahasan

Pada baris 1 dan 2 adalah pemanggilan library. Gunanya agar kode di file ini dapat menggunakan fungsi yang tersedia di `double_linked_list.h` dan `iostream`. Baris 3 gunanya efisiensi pengetikan.

Pada baris 6 adalah deklarasi struct double linked list dengan nama `dll`. Gunanya untuk mendeklarasi struct agar variabel dan fungsi di dalam struct bisa digunakan untuk kode program soal 2. Baris 7 digunakan untuk mendeklarasi variabel `N` (untuk jumlah char) dan `R` (ronde putar). Baris 8 digunakan untuk deklarasi variabel `C` bertipe char. Baris 9 digunakan untuk inisialisasi pointer dalam double linked list dengan memanggil fungsi `init` yang ada dalam struct. Pada baris 11 digunakan untuk input user, lalu nilai input itu dimasukkan ke variabel `N` dan `R`.

Pada baris 12 sampai 15 adalah loop yang digunakan untuk mengambil nilai input user dan memasukkannya ke linked list dengan fungsi `add_back` pada double linked list. Baris 16 digunakan untuk deklarasi array `X` dan `R`, lalu baris 17 sampai 19 adalah loop untuk input user dan nilainya akan disimpan dalam array `X` (lompatan alpha) dan `R` (lompatan beta).

Pada baris 22 dan 23 adalah inisialisasi pointer `alpha` dan `beta` untuk menunjuk ke head dan tail linked list. Lalu baris 24 sampai 49 adalah inti jawaban soal. Cara kerjanya dimulai dari iterasi luar (`i`) untuk perputaran sebanyak `R` ronde, lalu selama `R` ronde ini ada iterasi `j` untuk traverse pointer `alpha` sampai `X[i]` dan iterasi `k` untuk traverse pointer `beta` sampai `Y[i]`. Traverse hingga nilai yang sudah diinput user tadi.

Pada baris 33 sampai 47 adalah penentu tindakan proyektor alpha dan beta. Jika alpha dan beta bersebelahan (setelah alpha ada beta atau sebelum alpha ada beta), maka swap nilai alpha dan beta dengan menyimpan dulu nilai beta pada variabel `swap_beta` dan nilai data beta diubah menjadi alpha dan nilai alpha diubah menjadi beta sebelumnya dari `swap_beta`. Jika alpha dan beta menunjuk node yang sama, maka hapus node tersebut dengan cara traverse pointer target hingga bertemu pointer alpha dan selama traverse itu nilai `idx` ditambah untuk mendapatkan indeks target node yang akan dihapus (karena fungsi `delete` memerlukan nilai integer indeks). Setelah traverse hingga alpha dan menambah increment `idx`, pointer alpha dipindah menunjuk ke node sesudah target dan pointer beta dipindah menunjuk ke node sebelum target karna node target akan dihapus. Lalu pada baris 47, pemanggilan fungsi untuk menghapus node `idx`. Terakhir, pemanggilan fungsi `display` untuk menampilkan sisa atau hasil node sesudah operasi tadi.

Kompleksitas ruang pada fungsi ini adalah $O(N + R)$ karena memakai linked list dan array untuk nilai `X[R]` dan `Y[R]`. Kompleksitas waktunya adalah $O(n^2)$ karena nested loop, iterasi luar yang bergantung pada nilai `R` dan iterasi dalam yang bergantung pada `X[i]`, `Y[i]`, dan `idx` pada while loop.

MODUL 4:

SORTING

BUBBLE SORT

A. Source Code

Tabel 13. Source Code Bubble Sort

```

1 void bubble_sort(std::vector<int>& data, Metrics& m) {
2     int n = data.size();
3     auto start = Clock::now();
4     m.comparisons = 0;
5     m.swaps = 0;
6
7     for (int i = 0; i < n; i++) {
8         for (int j = 0; j < n - i - 1; j++) {
9             if (data[j] > data[j+1]) {
10                swap(data[j], data[j+1]);
11                m.swaps++;
12            }
13            m.comparisons++;
14        }
15    }
16
17    m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
18 }

```

B. Output Program

>> Bubble Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Bubble Sort	Random	100	4950	2564	0	0	0	0.026600
Bubble Sort	Sorted	100	4950	0	0	0	0	0.008200
Bubble Sort	Reverse Sorted	100	4950	4950	0	0	0	0.018100
Bubble Sort	Nearly Sorted	100	4950	296	0	0	0	0.010000
Bubble Sort	Many Duplicates	100	4950	2315	0	0	0	0.033700
Bubble Sort	Random	1000	499500	243190	0	0	0	1.231000
Bubble Sort	Sorted	1000	499500	0	0	0	0	0.481800
Bubble Sort	Reverse Sorted	1000	499500	499500	0	0	0	1.922200
Bubble Sort	Nearly Sorted	1000	499500	25778	0	0	0	0.556100
Bubble Sort	Many Duplicates	1000	499500	240808	0	0	0	0.958600
Bubble Sort	Random	10000	49995000	25118148	0	0	0	103.403100
Bubble Sort	Sorted	10000	49995000	0	0	0	0	16.362000
Bubble Sort	Reverse Sorted	10000	49995000	49995000	0	0	0	122.017400
Bubble Sort	Nearly Sorted	10000	49995000	3119352	0	0	0	42.082200
Bubble Sort	Many Duplicates	10000	49995000	25093086	0	0	0	90.979800

Gambar 7. Screenshot Jawaban Bubble Sort

C. Pembahasan

1. Kompleksitas waktu

- Best case = $O(n)$, karena data sudah hampir terurut atau bahkan sudah terurut, sehingga tidak perlu melakukan banyak looping dan swap.
- Average case = $O(n^2)$, karena datanya acak, jadi memakan waktu untuk looping berkali-kali dan swap berkali-kali.
- Worst case = $O(n^2)$, karena program jadi looping dan swap sebanyak mungkin. Contohnya pada reverse sorted, ia memiliki running time paling tinggi diantara data yang lain dengan jumlah yang sama.

2. Karakteristik algoritma

- Algoritma ini stabil, karna workflownya hanya swap-swap dengan elemen disampingnya. Tidak mengubah posisi elemen yang sudah benar.
- Algoritma ini termasuk in-place, karna swap dilakukan di dalam array yang sama.
- Mekanisme utamanya adalah melakukan swap elemen bersebelahan jika $[j]$ lebih besar dari $[j+1]$. Datanya terurut mulai dari belakang, dari data paling besar hingga terkecil.

3. Analisis operasi

- Jumlah comparison tergantung banyaknya data, semakin banyak data, semakin banyak algoritma ini melakukan comparison.
- Jumlah swap tergantung pada bentuk data, pada best case, tentunya swapnya 0 karena data sudah sorted, tapi di worst case swapnya paling banyak dan sangat banyak.

SELECTION SORT

A. Source Code

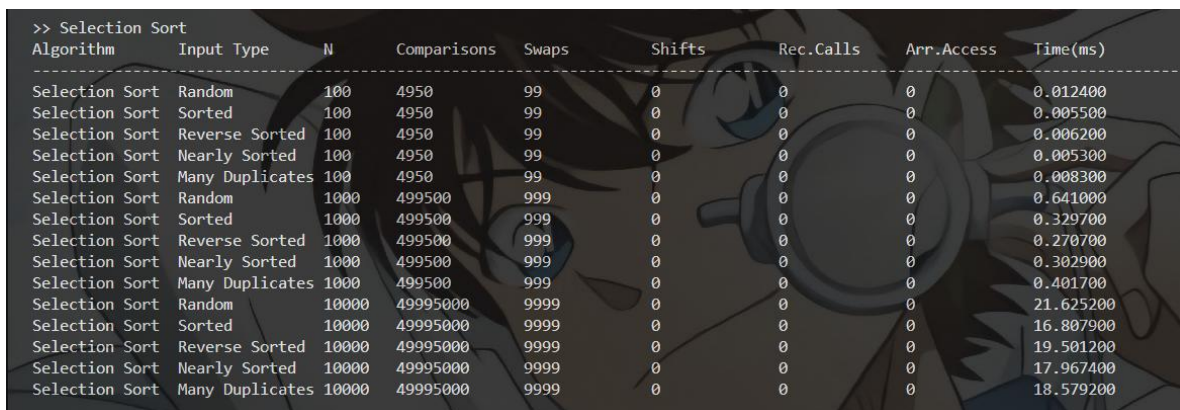
Tabel 14. Source Code Selection Sort

```

1 void selection_sort(std::vector<int>& data, Metrics& m)
2 {
3     int n = data.size();
4     auto start = Clock::now();
5     m.comparisons = 0;
6     m.swaps = 0;
7
8     for (int i = 0; i < n-1; i++) {
9         int minVal = i;
10        for (int j = i+1; j < n; j++) {
11            if (data[j] < data[minVal]) {
12                minVal = j;
13            }
14            m.comparisons++;
15        }
16        swap(data[i], data[minVal]);
17        m.swaps++;
18    }
19
20    m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
21 }

```

B. Output Program



>> Selection Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Selection Sort	Random	100	4950	99	0	0	0	0.012400
Selection Sort	Sorted	100	4950	99	0	0	0	0.005500
Selection Sort	Reverse Sorted	100	4950	99	0	0	0	0.006200
Selection Sort	Nearly Sorted	100	4950	99	0	0	0	0.005300
Selection Sort	Many Duplicates	100	4950	99	0	0	0	0.008300
Selection Sort	Random	1000	499500	999	0	0	0	0.641000
Selection Sort	Sorted	1000	499500	999	0	0	0	0.329700
Selection Sort	Reverse Sorted	1000	499500	999	0	0	0	0.270700
Selection Sort	Nearly Sorted	1000	499500	999	0	0	0	0.302900
Selection Sort	Many Duplicates	1000	499500	999	0	0	0	0.401700
Selection Sort	Random	10000	49995000	9999	0	0	0	21.625200
Selection Sort	Sorted	10000	49995000	9999	0	0	0	16.807900
Selection Sort	Reverse Sorted	10000	49995000	9999	0	0	0	19.501200
Selection Sort	Nearly Sorted	10000	49995000	9999	0	0	0	17.967400
Selection Sort	Many Duplicates	10000	49995000	9999	0	0	0	18.579200

Gambar 8. Screenshot Jawaban Selection Sort

C. Pembahasan

1. Kompleksitas waktu

- Best case = $O(n^2)$, karena meskipun sudah sorted, algoritma tetap looping untuk mencari elemen terkecil.
- Average case = $O(n^2)$, karena datanya acak, jadi memakan waktu untuk looping berkali-kali dan swap.
- Worst case = $O(n^2)$, karena program jadi looping dan swap sebanyak mungkin, ini sangat memakan waktu.

2. Karakteristik algoritma

- Algoritma ini tidak stabil, karna dengan swap elemen yang jauh, bisa menukar posisi relatif data duplikat dan bisa saja posisi elemen yang harusnya di depan jadi terkebelakang sementara karena swap tadi.
- Algoritma ini termasuk in-place, karna swap dilakukan di dalam array yang sama.
- Mekanisme utamanya adalah program menelusuri array untuk mencari elemen paling kecil, lalu jika sudah ketemu, masuk variabel minVal, elemen ini akan di swap ke [i]. Sehingga pengurutan data terjadi dari depan.

3. Analisis operasi

- Jumlah comparison tergantung banyaknya data, semakin banyak data, semakin banyak algoritma ini melakukan comparison antara data[i] dengan minVal.
- Jumlah swap tergantung pada banyaknya data. Walau sorted, program menukar elemen dengan dirinya sendiri, karena data[i] di compare dengan variabel minVal.

INSERTION SORT

A. Source Code

Tabel 15. Source Code Insertion Sort

```

1 void insertion_sort(std::vector<int>& data, Metrics& m)
  {
2     int n = data.size();
3     auto start = Clock::now();
4     m.comparisons = 0;
5     m.shifts = 0;
6
7     for (int i = 0; i < n; i++) {
8         int key = data[i];
9         int j = i - 1;
10        while (j >= 0 && data[j] > key) {
11            data[j+1] = data[j];
12            j--;
13            m.comparisons++;
14        }
15        data[j+1] = key;
16        m.shifts++;
17    }
18
19    m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
20 }

```

B. Output Program

>> Insertion Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Insertion Sort	Random	100	2564	0	100	0	0	0.007800
Insertion Sort	Sorted	100	0	0	100	0	0	0.000800
Insertion Sort	Reverse Sorted	100	4950	0	100	0	0	0.012100
Insertion Sort	Nearly Sorted	100	296	0	100	0	0	0.001400
Insertion Sort	Many Duplicates	100	2315	0	100	0	0	0.006300
Insertion Sort	Random	1000	243190	0	1000	0	0	0.538000
Insertion Sort	Sorted	1000	0	0	1000	0	0	0.002300
Insertion Sort	Reverse Sorted	1000	499500	0	1000	0	0	1.231100
Insertion Sort	Nearly Sorted	1000	25778	0	1000	0	0	0.058500
Insertion Sort	Many Duplicates	1000	240808	0	1000	0	0	0.718500
Insertion Sort	Random	10000	25118148	0	10000	0	0	57.560800
Insertion Sort	Sorted	10000	0	0	10000	0	0	0.017600
Insertion Sort	Reverse Sorted	10000	49995000	0	10000	0	0	120.365400
Insertion Sort	Nearly Sorted	10000	3119352	0	10000	0	0	8.238100
Insertion Sort	Many Duplicates	10000	25093086	0	10000	0	0	62.463700

Gambar 9. Screenshot Jawaban Insertion Sort

C. Pembahasan

1. Kompleksitas waktu

- Best case = $O(n)$, karena setiap elemen dibandingkan hanya sekali, lalu di insert ke posisi yang seharusnya.
- Average case = $O(n^2)$, karena datanya acak, jadi memakan waktu untuk looping dan geser elemen untuk insert.
- Worst case = $O(n^2)$, karena program jadi looping sebanyak mungkin untuk compare dan shift.

2. Karakteristik algoritma

- Algoritma ini stabil, karna proses shift berhenti jika menemukan elemen dengan nilai yang sama atau posisi elemen sudah tepat.
- Algoritma ini termasuk in-place, karna shifts dilakukan di dalam array yang sama.
- Mekanisme utamanya adalah mengambil nilai key di $[j+1]$, lalu dibandingkan dengan elemen di sebelah kirinya (j), jika lebih besar, maka geser nilainya ke kanan. Hingga key menemukan posisinya yang tepat, lalu nilai key ditaruh di indeks yang tepat itu.

3. Analisis operasi

- Jumlah comparison tergantung banyaknya data, semakin banyak data, semakin banyak algoritma ini melakukan comparison antara $data[j]$ dengan key.
- Jumlah shifts tergantung pada banyaknya data. Walau data sorted, tetap ada shift karena nilai data di indeks $j+1$ disimpan ke variabel key. Dan total shift sama dengan jumlah data karena nilai data dari indeks 0 sampai N disimpan ke key secara bergantian gitu.

MERGE SORT

A. Source Code

Tabel 16. Source Code Merge Sort

1	void merge(std::vector<int>& data, int left, int mid,
2	int right, Metrics& m) {
3	int n1 = mid - left + 1;
4	int n2 = right - mid;
5	m.comparisons = 0;
6	std::vector<int> L(n1), R(n2);
7	
8	for (int i = 0; i < n1; i++) {
9	L[i] = data[left+i];
10	}
11	for (int j = 0; j < n2; j++) {
12	R[j] = data[mid + 1 + j];
13	}
14	
15	int i = 0, j = 0, k = left;
16	
17	while (i < n1 && j < n2) {
18	if (L[i] <= R[j]) {
19	data[k] = L[i];
20	i++;
21	} else {
22	data[k] = R[j];
23	j++;
24	}
25	k++;
26	m.comparisons++;
27	}
28	
29	while (i < n1) {
30	data[k] = L[i];
31	i++;
32	k++;
33	}
34	
35	while (j < n2) {
36	data[k] = R[j];
37	j++;
38	k++;
39	}
40	}
41	


```

42 void mergeSort(std::vector<int>& data, int left, int
    right, Metrics& m) {
43     m.comparisons = 0;
44     m.recursive_calls = 0;
45     if (left >= right) {
46         m.comparisons++;
47         return;
48     }
49
50     int mid = left + (right - left) / 2;
51     mergeSort(data, left, mid, m);
52     m.recursive_calls++;
53     mergeSort(data, mid+1, right, m);
54     m.recursive_calls++;
55     merge(data, left, mid, right, m);
56 }
57
58 void merge_sort(std::vector<int>& data, Metrics& m) {
59     auto start = Clock::now();
60
61     mergeSort(data, 0, data.size()-1, m);
62
63     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
64 }

```

B. Output Program

>> Merge Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Merge Sort	Random	100	99	0	0	6	0	0.046500
Merge Sort	Sorted	100	50	0	0	6	0	0.045100
Merge Sort	Reverse Sorted	100	50	0	0	6	0	0.048900
Merge Sort	Nearly Sorted	100	96	0	0	6	0	0.029400
Merge Sort	Many Duplicates	100	95	0	0	6	0	0.054600
Merge Sort	Random	1000	998	0	0	9	0	0.264800
Merge Sort	Sorted	1000	500	0	0	9	0	0.159200
Merge Sort	Reverse Sorted	1000	500	0	0	9	0	0.280000
Merge Sort	Nearly Sorted	1000	993	0	0	9	0	0.311000
Merge Sort	Many Duplicates	1000	995	0	0	9	0	0.229000
Merge Sort	Random	10000	9998	0	0	13	0	2.278100
Merge Sort	Sorted	10000	5000	0	0	13	0	1.348500
Merge Sort	Reverse Sorted	10000	5000	0	0	13	0	2.319300
Merge Sort	Nearly Sorted	10000	9978	0	0	13	0	2.341100
Merge Sort	Many Duplicates	10000	9997	0	0	13	0	2.005100

Gambar 10. Screenshot Jawaban Merge Sort

C. Pembahasan

1. Kompleksitas waktu

- Best case, average case, worst case tetap $O(n \log n)$, karena array selalu dibagi jadi 2 bagian hingga terkecil.
2. Karakteristik algoritma
- Algoritma ini stabil, karna tidak memindah posisi relatif elemen, yang kiri tetaplah di kiri.
 - Algoritma ini termasuk out-of-place, karna perlu array tambahan untuk menyimpan hasil penggabungan elemen yang dibagi 2 tadi.
 - Mekanisme utamanya adalah array dibagi 2, lalu rekursif dan jadi terbagi 2 lagi hingga sisa 1 elemen. Lalu bagian-bagian tersebut digabungkan secara berurutan di array lain.
3. Analisis operasi
- Jumlah comparison tergantung pada bentuk dan banyaknya data, karna array selalu dibagi 2, jadi kalau datanya sorted, hanya pointer elemen kiri yang bergerak, membandingkan dengan elemen pertama di kanan. Kalau tidak sorted, pointer elemen kanan juga naik jika elemen sebelumnya dimasukkan ke array baru.
 - Jumlah recursive calls tergantung pada banyaknya data, karena array selalu dibagi 2. Misalnya $N = 100$ selalu 6 recursive calls karna 6 kali membagi array menjadi 2 hingga tersisa 1 elemen di array.

QUICK SORT

A. Source Code

Tabel 17. Source Code Quick Sort

```
1  int partition(std::vector<int>& data, Metrics& m, int
   low, int high) {
2      int pivot = data[high];
3      int i = low - 1;
4      m.swaps = 0;
5
6      for (int j = low; j <= high - 1; j++) {
7          if (data[j] < pivot) {
8              i++;
9              swap(data[i], data[j]);
10             m.swaps++;
11         }
12     }
13
14     swap(data[i + 1], data[high]);
15     m.swaps++;
16     return i + 1;
17 }
18
19 void quickSort(std::vector<int>& data, Metrics& m, int
   low, int high) {
20     m.comparisons = 0;
21     if (low < high) {
22         int parti = partition(data, m, low, high);
23         m.comparisons++;
24
25         quickSort(data, m, low, parti-1);
26         m.recursive_calls++;
27         quickSort(data, m, parti+1, high);
28         m.recursive_calls++;
29     }
30 }
31
32 void quick_sort(std::vector<int>& data, Metrics& m) {
33     auto start = Clock::now();
34
35     quickSort(data, m, 0, data.size()-1);
36
37     m.time_ms = std::chrono::duration<double,
   std::milli>(Clock::now() - start).count();
38 }
```

B. Output Program

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Quick Sort	Random	100	64	394	0	128	0	0.003700
Quick Sort	Sorted	100	99	5049	0	198	0	0.004100
Quick Sort	Reverse Sorted	100	99	2549	0	198	0	0.004300
Quick Sort	Nearly Sorted	100	95	2607	0	190	0	0.003200
Quick Sort	Many Duplicates	100	90	236	0	180	0	0.003000
Quick Sort	Random	1000	670	6125	0	1340	0	0.034800
Quick Sort	Sorted	1000	999	500499	0	1998	0	0.239300
Quick Sort	Reverse Sorted	1000	999	250499	0	1998	0	0.177600
Quick Sort	Nearly Sorted	1000	805	25081	0	1610	0	0.028300
Quick Sort	Many Duplicates	1000	900	4920	0	1800	0	0.031800
Quick Sort	Random	10000	6657	77996	0	13314	0	0.471100
Quick Sort	Sorted	10000	9999	50004999	0	19998	0	21.383900
Quick Sort	Reverse Sorted	10000	9999	25004999	0	19998	0	15.684500
Quick Sort	Nearly Sorted	10000	8255	458547	0	16510	0	0.404500
Quick Sort	Many Duplicates	10000	9001	67083	0	18002	0	0.405700

Gambar 11. Screenshot Jawaban Quick Sort

C. Pembahasan

1. Kompleksitas waktu

- Best case = $O(n \log n)$, karna pivot yang dipilih untuk membagi array menjadi 2 sama besar.
- Average case = $O(n \log n)$, jika pemilihan pivot untuk membagi 2 arraynya tepat.
- Worst case = $O(n^2)$, jika salah pilih pivot, misalnya pivotnya di elemen pertama atau terakhir pada data sorted (ascending atau descending). Sehingga pembagian tidak seimbang.

2. Karakteristik algoritma

- Algoritma ini tidak stabil, karna pemindahan elemen sisi kiri dan kanan pivot yang berjauhan, memungkinkan memindah posisi relatif elemen yang nilainya sama.
- Algoritma ini termasuk in-place, karna partisi dilakukan di array yang sama (tapi perlu memori untuk call stack).
- Mekanisme utamanya adalah array dibagi 2, di kiri pivot dan kanan pivot. Lalu semua elemen yang lebih kecil dari pivot dipindah ke kiri, elemen lebih besar dari pivot dipindah ke kanan.

3. Analisis operasi

- Jumlah comparison tergantung pada partisi array, karna setiap elemen di kiri dan kanan pivot harus dibandingkan dengan nilai pivot, agar elemen mengetahui posisinya.

- Jumlah swap tergantung pada banyaknya elemen yang salah posisi, karna swap dilakukan jika elemen sisi kiri yang harusnya di kanan atau elemen kanan yang harusnya di kiri.
- Jumlah recursive calls tergantung pada banyaknya data, karena array selalu dibagi-bagi menjadi 2, dan juga tergantung pivotnya seimbang atau tidak.

RADIX SORT

A. Source Code

Tabel 18. Source Code Radix Sort

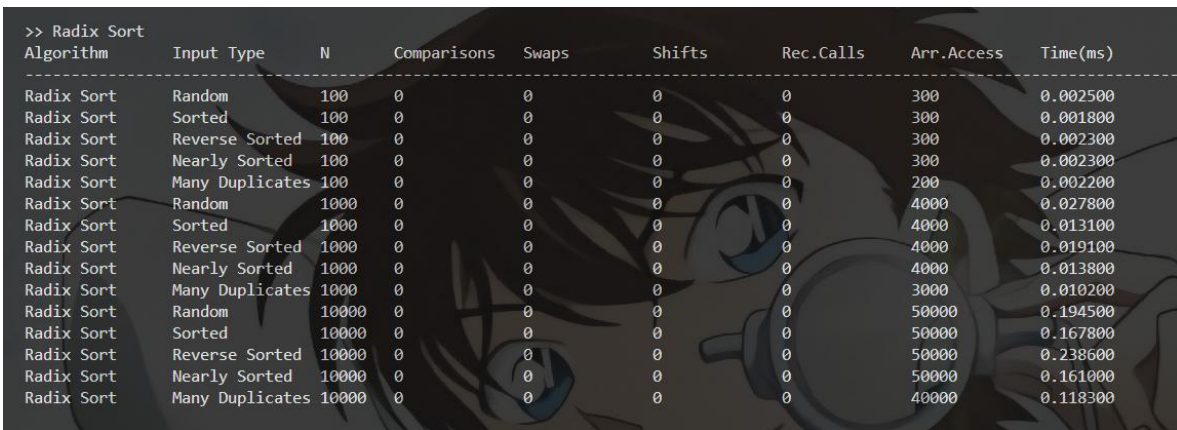
1	int getMax(std::vector<int>& data, int n) {
2	int max = data[0];
3	for (int i = 0; i < n; i++) {
4	if (data[i] > max) {
5	max = data[i];
6	}
7	}
8	return max;
9	}
10	
11	void count_sort(std::vector<int>& data, Metrics& m, int
	n, int exp) {
12	int output[n];
13	int i, count[10] = { 0 };
14	m.array_accesses = 0;
15	
16	for (i = 0; i < n; i++) {
17	count[(data[i] / exp) % 10]++;
18	}
19	
20	for (i = n-1; i >= 0; i--) {
21	output[count[(data[i] / exp) % 10] - 1] =
	data[i];
22	count[(data[i] / exp) % 10]--;
23	}
24	
25	for (i = 0; i < n; i++) {
26	data[i] = output[i];
27	m.array_accesses++;
28	}
29	}
30	
31	void radixSort (std::vector<int>& data, Metrics& m, int
	n) {
32	int maxVal = getMax(data, n);
33	
34	for (int exp = 1; (maxVal / exp) > 0; exp *= 10) {
35	count_sort(data, m, n, exp);
36	}
37	}
38	
39	void radix_sort(std::vector<int>& data, Metrics& m) {

```

40     if (data.empty()) return;
41     auto start = Clock::now();
42     int n = sizeof(data) / sizeof(data[0]);
43
44     radixSort(data, m, n);
45
46     m.time_ms      =      std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
47 }

```

B. Output Program



>> Radix Sort								
Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Radix Sort	Random	100	0	0	0	0	300	0.002500
Radix Sort	Sorted	100	0	0	0	0	300	0.001800
Radix Sort	Reverse Sorted	100	0	0	0	0	300	0.002300
Radix Sort	Nearly Sorted	100	0	0	0	0	300	0.002300
Radix Sort	Many Duplicates	100	0	0	0	0	200	0.002200
Radix Sort	Random	1000	0	0	0	0	4000	0.027800
Radix Sort	Sorted	1000	0	0	0	0	4000	0.013100
Radix Sort	Reverse Sorted	1000	0	0	0	0	4000	0.019100
Radix Sort	Nearly Sorted	1000	0	0	0	0	4000	0.013800
Radix Sort	Many Duplicates	1000	0	0	0	0	3000	0.010200
Radix Sort	Random	10000	0	0	0	0	50000	0.194500
Radix Sort	Sorted	10000	0	0	0	0	50000	0.167800
Radix Sort	Reverse Sorted	10000	0	0	0	0	50000	0.238600
Radix Sort	Nearly Sorted	10000	0	0	0	0	50000	0.161000
Radix Sort	Many Duplicates	10000	0	0	0	0	40000	0.118300

Gambar 12. Screenshot Jawaban Radix Sort

C. Pembahasan

1. Kompleksitas waktu

- Best case, average case, worst case tetap $O(d * (N + k))$, d adalah jumlah digit, N jumlah elemen, dan k adalah bucket representasi angka. Karena running time dipengaruhi oleh ketiga komponen tersebut.

2. Karakteristik algoritma

- Algoritma ini stabil, karna workflownya meletakkan nilai ke bucket sesuai digit.
- Algoritma ini termasuk out-of-place, karna adanya bucket, menyediakan tempat untuk menghitung elemen-elemen dengan nilai digit yang sama.
- Mekanisme utamanya adalah membuat bucket, 10 ruang. Lalu elemen dengan digit sekian, disimpan ke bucket indeks ke sekian. Dimulai dari satuan, puluhan, dan seterusnya.

3. Analisis operasi

- Jumlah array access bergantung pada banyaknya data dan jumlah digit angka terbesarnya. Karena sistemnya bukan comparison, swap, dan sebagainya, tapi memang sistem collecting elemen yang digitnya sesuai dengan indeks bucket yang disediakan. Semakin banyak data, semakin banyak array access.

ANALISIS PERFORMA SORTING

Perbandingan algoritma

1. Bubble sort, paling lambat, banyak comparison dan swap. Running time melonjak seiring bertambahnya data dan perbedaan bentuk data. Cocok untuk dataset kecil, tapi jarang digunakan.
2. Selection sort, lebih baik dari bubble sort, karna total swap berkurang, tapi lebih lambat dari insertion sort. Running time naik drastis seiring banyaknya data, beda dengan bubble sort. Cocok untuk dataset kecil, tapi mending insertion sort.
3. Insertion sort, lebih baik dari bubble dan selection, tapi kompleksitas masih belum efisien. Sangat cocok untuk dataset kecil.
4. Merge sort, algoritma stabil dan cepat. Walaupun mengakses memori yang lumayan. Cocok untuk dataset besar, terutama yang mengutamakan stabilitas.
5. Quick sort, algoritma paling cepat, namun tergantung pemilihan pivot yang baik. Sangat cocok untuk dataset besar, selain sangat cepat, juga hemat memori.
6. Radix sort, sangat cepat, namun memerlukan tambahan memori yang sangat besar. Cocok untuk dataset dengan tipe bilangan bulat saja.

Pengaruh distribusi data

1. Untuk sorted array, menjadikan bubble sort dan insertion sort sangat cepat. Namun menjadi worst case untuk quick sort.
2. Untuk nearly sorted, menjadikan bubble sort dan insertion sort menjadi efisien, cocok digunakan.
3. Untuk data random, bagus bagi quick sort dan merge sort.
4. Untuk data bilangan bulat berdigit, bagusnya pakai radix sort, paling bagus.

Hubungan teori kompleksitas dengan eksperimen

Hasil eksperimen menunjukkan pertumbuhan waktu pada masing-masing sorting dan semuanya sesuai. Ada yang naik drastis, ada yang stabil, ada yang lurus, dan sebagainya. Hasil eksperimen di dunia nyata (benchmark) mengkonfirmasi tren yang diprediksi oleh teori kompleksitas (Big-O).

MODUL 5: SEARCHING

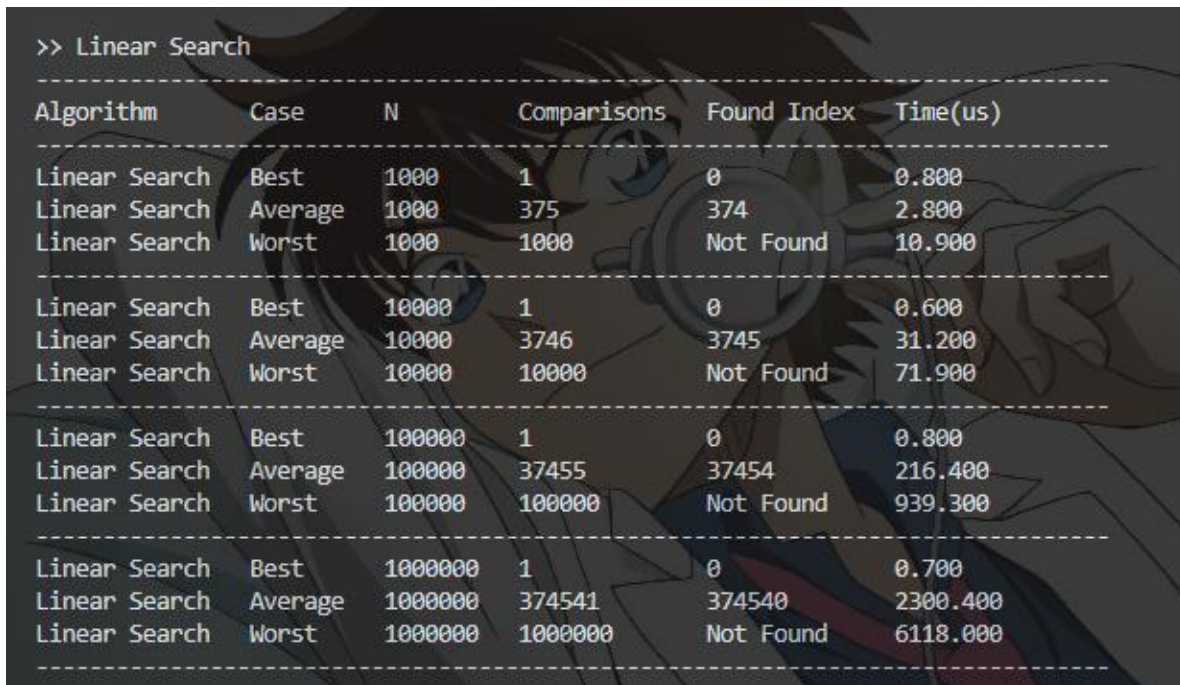
LINEAR SEARCH

A. Source Code

Tabel 19. Source Code Linear Search

1	<code>void linear_search(const std::vector<int>& data, int</code>
	<code>target, Metrics& m) {</code>
2	<code> auto start = Clock::now();</code>
3	
4	<code> m.comparisons = 0;</code>
5	<code> for (int i = 0; i < data.size(); i++) {</code>
6	<code> m.comparisons++;</code>
7	<code> if(data[i] == target) {</code>
8	<code> m.found_index = i;</code>
9	<code> break;</code>
10	<code> } else {</code>
11	<code> m.found_index = -1;</code>
12	<code> }</code>
13	<code> }</code>
14	<code> m.time_us = std::chrono::duration<double,</code>
	<code>std::micro>(Clock::now() - start).count();</code>
15	<code>}</code>

B. Output Program



```
>> Linear Search
```

Algorithm	Case	N	Comparisons	Found Index	Time(us)
Linear Search	Best	1000	1	0	0.800
Linear Search	Average	1000	375	374	2.800
Linear Search	Worst	1000	1000	Not Found	10.900
Linear Search	Best	10000	1	0	0.600
Linear Search	Average	10000	3746	3745	31.200
Linear Search	Worst	10000	10000	Not Found	71.900
Linear Search	Best	100000	1	0	0.800
Linear Search	Average	100000	37455	37454	216.400
Linear Search	Worst	100000	100000	Not Found	939.300
Linear Search	Best	1000000	1	0	0.700
Linear Search	Average	1000000	374541	374540	2300.400
Linear Search	Worst	1000000	1000000	Not Found	6118.000

Gambar 13. Screenshot Jawaban Linear Search

C. Pembahasan

Kode tersebut adalah implementasi linear search, dengan pendekatan brute force. Cara kerjanya yaitu dengan for loop, untuk mencari target dan membandingkan semua data satu persatu dengan target. Kalau target ketemu, loop di break. Kalau target tidak ketemu, nilai m.found_index jadi -1. Algoritma ini tidak memerlukan data terurut, karena semua data akan di-compare dengan target secara linear. Algoritma ini bersifat iteratif dan dapat diimplementasikan dengan rekursif, tapi kalau rekursif, algoritma ini akan memakan ruang yang banyak untuk call stack, hingga kompleksitas ruangnya $O(n)$. Selain itu, kalau dengan rekursif, bisa terjadi stack overflow jika data terlalu banyak.

Untuk best case, algoritma langsung menemukan target di elemen pertama, kompleksitas waktunya jadi $O(1)$, tidak perlu loop sampai N. Average case, algoritma melakukan loop dan compare sampai target, kompleksitas waktunya $O(n)$. Worst case, target tidak berada dalam data tersebut. Algoritma tetap melakukan loop sampai N sehingga kompleksitas waktunya $O(n)$. Untuk semua case, linear search langsung loop dan mengakses array yang ada, tidak membuat array baru, sehingga kompleksitas ruangnya $O(1)$.

Jumlah comparison tergantung pada letak target, karena ini brute force, semuanya di compare. Jika target berada di elemen pertama, cukup sekali compare. Semakin banyak jumlah comparison, semakin jauh letak elemen target. Nilai indeks found tergantung dimana letak elemen target. Kalau 0 berarti target ada di elemen pertama, kalau 374 berarti target berada di elemen 375, kalau -1 berarti target tidak ditemukan. Ukuran data mempengaruhi running time dan jumlah comparison jika bukan best case atau jika target bukan di elemen pertama. Semakin banyak data dan semakin jauh elemen target, jumlah comparison dan running time semakin tinggi.

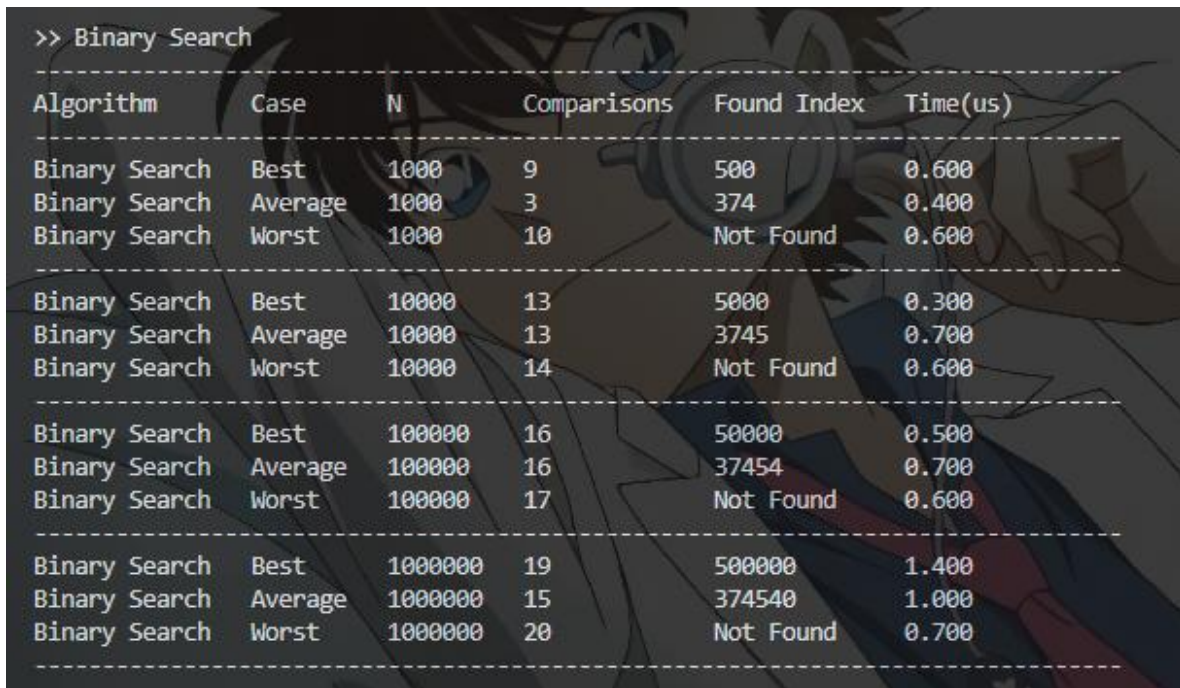
BINARY SEARCH

A. Source Code

Tabel 20. Source Code Binary Search

1	void binary_search(const std::vector<int>& data, int
2	target, Metrics& m) {
3	auto start = Clock::now();
4	int low = 0;
5	int high = data.size() - 1;
6	m.comparisons = 0;
7	
8	while (low <= high) {
9	int mid = low + (high - low) / 2;
10	
11	if (data[mid] == target) {
12	m.found_index = mid;
13	break;
14	} else {
15	m.found_index = -1;
16	}
17	if (data[mid] < target) {
18	low = mid + 1;
19	} else {
20	high = mid - 1;
21	}
22	m.comparisons++;
23	}
24	m.time_us = std::chrono::duration<double,
25	std::micro>(Clock::now() - start).count();
	}

B. Output Program



```
>> Binary Search
```

Algorithm	Case	N	Comparisons	Found Index	Time(us)
Binary Search	Best	1000	9	500	0.600
Binary Search	Average	1000	3	374	0.400
Binary Search	Worst	1000	10	Not Found	0.600
Binary Search	Best	10000	13	5000	0.300
Binary Search	Average	10000	13	3745	0.700
Binary Search	Worst	10000	14	Not Found	0.600
Binary Search	Best	100000	16	50000	0.500
Binary Search	Average	100000	16	37454	0.700
Binary Search	Worst	100000	17	Not Found	0.600
Binary Search	Best	1000000	19	500000	1.400
Binary Search	Average	1000000	15	374540	1.000
Binary Search	Worst	1000000	20	Not Found	0.700

Gambar 14. Screenshot Jawaban Binary Search

C. Pembahasan

Kode tersebut adalah implementasi binary search, metode serach dengan pendekatan divide and conquer. Cara kerjanya yaitu dengan while loop, selama nilai low tidak melebihi high. Algoritma langsung meloncat ke nilai mid, comparing nilai mid dengan target. Kalau nilai target lebih kecil dari mid, maka nilai high diubah ke $\text{mid} - 1$ agar algoritma mencari nilai mid lagi di sisi kiri mid sebelumnya. Kalau target lebih besar dari mid, nilai low diubah ke $\text{mid} + 1$ agar algoritma mencari nilai mid di sisi kanan mid sebelumnya. Search terus dilakukan hingga target ketemu tepat di mid dan loop di break. Kalau tidak ketemu, nilai `m.found_indeks` diisi -1. Algoritma ini memerlukan data terurut, karena agar algoritma tahu search dilanjutkan di sisi kiri mid (ketika target lebih kecil dari mid) atau di sisi kanan mid (ketika target lebih besar dari mid). Algoritma ini bersifat iteratif dan dapat diimplementasikan dengan rekursif, karena paradigmanya sama dengan merge sort, tetapi kalau memakai algoritma rekursif, kompleksitas ruangnya menjadi $O(\log n)$. Sehingga masih lebih bagus menggunakan algoritma iteratif (`i guess`).

Untuk best case, posisi target di tengah. Algoritma langsung menemukan datanya dalam 1 kali compare, karena elemen target persis di mid. Average case, posisi target bukan ditengah, algoritma akan mencari langsung ke tengah (mid), lalu jika target bukan di tengah, algoritma akan membuat nilai tengah lain, di sisi kanan mid sebelumnya atau di sisi kiri mid. Hal ini terus berulang hingga $\text{mid} == \text{target}$. Sehingga kompleksitas waktunya $O(n \log n)$, memiliki paradigma divide and conquer. Worst case, target tidak berada dalam data tersebut. Algoritma tetap menjalankan while loop hingga nilai low lebih tinggi dari high. Karena ini divide and conquer, kompleksitas waktunya tetap $O(n \log n)$. Untuk semua case, binary search langsung loop dan mengakses array yang ada, tidak membuat array baru, hanya menambah variabel low, mid, high. Sehingga kompleksitas ruangnya $O(1)$.

Jumlah comparison tergantung pada letak target, semakin dekat dengan mid atau low atau high, semakin tinggi jumlah comparison, karna ia membagi-bagi terus sampai ketemu pas di mid. Misalnya 51 dari 100, maka algoritma ke 50 dulu, lalu 75, lalu 62, lalu 56, 53, hingga akhirnya 51. Nilai indeks found tergantung dimana letak elemen target. Kalau 500 berarti target ada di elemen ke 501, kalau -1 berarti target tidak ditemukan. Ukuran data mempengaruhi running time dan jumlah comparison. Semakin banyak data dan semakin dekat elemen target dengan low/mid/high, jumlah comparison dan running time semakin tinggi.

ANALISIS PERFORMA SEARCHING

Perbandingan running time antar algoritma jelas lebih singkat binary search, untuk data 1 juta pun hanya memakan 0.7 sampai 1.4 microsecond. Sedangkan linear search jauh diatas itu, untuk best case hanya 0.3 microsecond, worst case mencapai 4178 microsecond. Binary search, dilihat dari running time untuk semua case yang tidak melebihi 2 microsecond. Sedangkan linear search apalagi worst case dengan data yang banyak, bisa mencapai 4 milisecond.

Algoritma linear search cocok untuk dataset kecil, selain itu juga tidak perlu data yang terurut karena brute force. Sedangkan binary search cocok untuk dataset besar, karena waktu yang diperlukan untuk searching jadi lebih singkat, namun data harus diurutkan dahulu.

Kondisi data yang terurut dan acak hanya memengaruhi linear search, jika elemen target itu kecil, maka waktu eksekusi makin singkat jika datanya sudah terurut. Namun jadi worst case jika targetnya 100 sedangkan jumlah datanya 100. Untuk binary search, datanya harus terurut, kalau tidak terurut, nilai mid akan salah dan target tidak akan ketemu.

MODUL 6:

GRAPH DAN TREE

SOAL 1

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge. Setiap edge ditulis sebagai U,V,W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U,V,W tidak otomatis berarti terdapat edge dari V menuju U . Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

M

$U_1 V_1 W_1$

$U_2 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

V1 V2 ... VN

V1 a11 a12 ... a1N

V2 a21 a22 ... a2N

...

VN aN1 aN2 ... aNN

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai

a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki self-loop.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

Penjelasan Contoh

Edge A B 4 menghasilkan nilai 4 pada baris A kolom B. Karena graph berarah, nilai pada baris B kolom A tetap 0 selama tidak ada edge dari B menuju A. Edge D C 3 menghasilkan nilai 3 pada baris D kolom C. Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0

A. Source Code

Tabel 21. Source Code File soal_1.cpp

```
1  #include <iostream>
2  using namespace std;
3  #define SIZE 10
4  #define NO_EDGE -1
5
6  typedef struct {
7      int adjMatrix[SIZE][SIZE];
8      char vertex[SIZE];
9  } Graph;
10
11
12 void init(Graph *g, int n) {
13     for (int i=0; i<n; i++) {
14         for (int j=0; j<n; j++) {
15             g->adjMatrix[i][j] = NO_EDGE;
16         }
17         g->vertex[i] = 0;
18     }
19 }
20
21 void addEdge(Graph *g, int u, int v, int w, int n) {
22     if (u >= 0 && u < n && v >= 0 && v < n) {
23         g->adjMatrix[u][v] = w;
24     }
25 }
26
27
28 void addVertex (Graph *g, int v, char namaVertex, int n)
29 {
30     if (v >= 0 && v < n) {
31         g->vertex[v] = namaVertex;
32     }
33 }
```

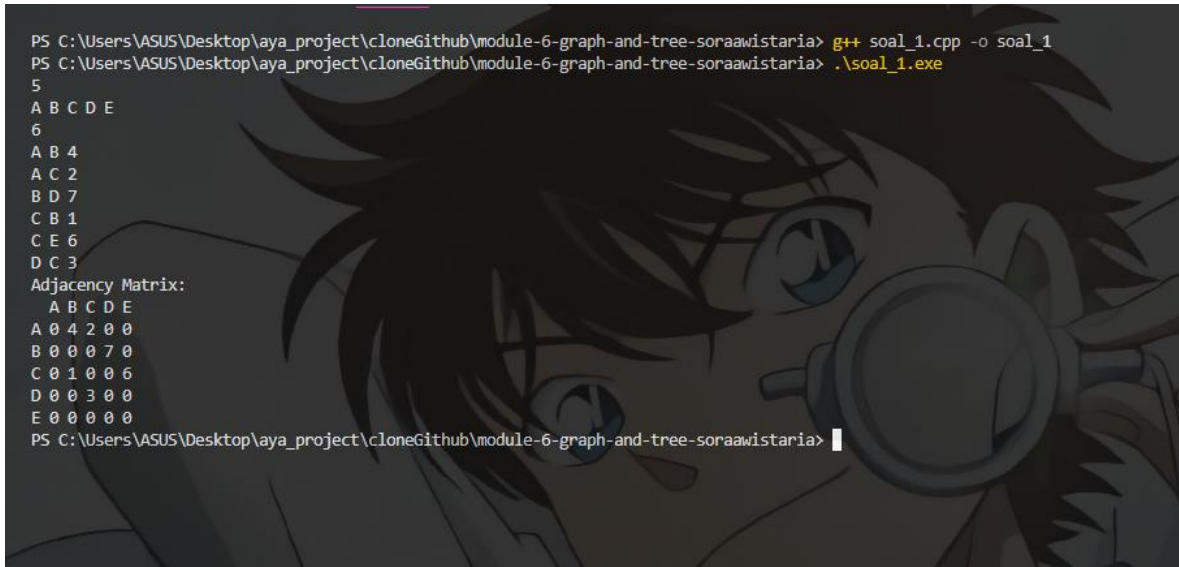
```

33
34 void printGraph (Graph *g, int n) {
35     cout << "Adjacency Matrix:\n";
36
37     cout << " ";
38     for (int i = 0; i < n; i++) {
39         cout << g->vertex[i];
40         if (i < n - 1) cout << " ";
41     }
42     cout << endl;
43
44     for (int i=0; i < n; i++) {
45         cout << g->vertex[i];
46         for (int j=0; j<n; j++) {
47             cout << " ";
48             if (g->adjMatrix[i][j] == NO_EDGE) {
49                 cout << '0';
50             } else {
51                 cout << g->adjMatrix[i][j];
52             }
53         }
54         cout << endl;
55     }
56 }
57
58
59 int main() {
60     Graph g;
61     int N, M;
62     cin >> N;
63     init(&g, N);
64
65
66     for (int i=0; i<N; i++) {
67         char vertex;
68         cin >> vertex;
69         addVertex(&g, i, vertex, N);
70     }
71
72     cin >> M;
73     char U, V;
74     int W;
75     for (int i=0; i<M; i++) {
76         cin >> U >> V >> W;
77         addEdge(&g, (U - 'A'), (V - 'A'), W, N);
78     }
79

```

80	printGraph(&g, N);
81	}

B. Output Program



```

PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> g++ soal_1.cpp -o soal_1
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> .\soal_1.exe
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
  A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria>

```

Gambar 15. Screenshot Jawaban Soal 1 Modul 6

C. Pembahasan

Kode berikut merupakan algoritma mengubah adjacency list menjadi adjacency matrix dengan graph. Jadi, inputnya berupa nama jumlah vertex, nama vertex, jumlah edge dan weight dari edge tersebut. Nantinya outputnya berupa graph yang sudah ada weighted edge. Disini ada 1 struct dan 4 fungsi selain fungsi main.

Pertama, struct `graph`, isinya variabel array adjacency matrix dengan ukuran maksimal 10x10 sesuai constraint, lalu variabel array vertex yang menyimpan nama-nama vertex yang diwakili abjad A-Z.

Kedua, fungsi `init graph`, untuk inisialisasi graph. Cara kerjanya dengan membuat array 2D seukuran $N \times N$, semuanya diisi -1 karna belum ada edge. Lalu setiap iterasi i , vertex diisi 0 karna nama vertexnya belum diinput. Intinya ini menyiapkan container. Time complexity-nya jadi $O(n^2)$ karna iterasi i dan j sampai N keduanya, space complexity juga $O(n^2)$ karena ini membuat array dengan ukuran $N \times N$ dan array vertex ukuran N . Jadi kalau N nya 5, perlu array ukuran 25 buat menyimpan graph dan weighted edge nya. Perlu array ukuran 5 juga buat nyimpan char vertex.

Ketiga, fungsi `addEdge`, buat nambah edge. Cara kerjanya dengan mengisi nilai w di indeks u ke v , kalau $0 \leq u < n$ dan $0 \leq v < n$. Jadi ya parameter fungsinya ada `struct graph`, u (vertex awal), v (vertex tujuan), w (weight edge), dan n (ukuran matrix). Time complexity-nya $O(1)$ karena dia langsung memasukkan edge ke graph yang sudah di inisialisasi, space complexity-nya $O(1)$ karena dia cuma nambah w ke graph yang sudah ada.

Keempat, fungsi `addVertex`, buat nambah vertex. Cara kerjanya dengan menambah char nama vertex hasil input ke array vertex yang sudah diinit sebelumnya. Dia nambah nama vertex kalau $0 \leq v < n$ agar tidak out of index. Time complexity-nya $O(1)$ karna dia langsung nambah nama vertex ke array yang sudah ada (tanpa loop lagi) dan space complexity-nya $O(1)$ karna dia cuma nambah char ke array yang sudah ada.

Kelima, fungsi `printGraph` buat mencetak graph yang sudah ada vertex, nama, dan edge nya. Cara kerjanya, baris pertama itu print kata adjacency matrix dulu, lalu baris kedua itu print nama-nama vertex, lalu baru print matrix ukuran $N \times N$ tadi. Time complexity-nya $O(N^2)$, $N + N^2$, pertama buat print nama vertex, lalu $N \times N$ buat print adjacency matrix.

Terakhir, fungsi `main`, fungsi utama tempat input dan output serta proses program. Pertama-tama ada deklarasi `graph`, N , M . Lalu input N . Lalu inisialisasi `graph`. Lalu loop 0 sampai N untuk input vertex. Lalu input M dan deklarasi variabel U , V , W . Lalu loop 0 sampai M (jumlah edge) untuk menambah edge ke graph. Terakhir memanggil fungsi `printGraph`. Time complexity-nya $O(N + M + N^2)$, karena loop input vertex, input edge-edge untuk graph, dan cetak output graph yang sampai N^2 .

SOAL 2

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik. Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j . Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

$A_{11} A_{12} \dots A_{1N}$

$A_{21} A_{22} \dots A_{2N}$

...

$A_{N1} A_{N2} \dots A_{NN}$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

$V_1 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

$V_2 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) E -> -

Penjelasan Contoh

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk A adalah (B,4), (C,2).

Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -.

A. Source Code

Tabel 22. Source Code File soal_2.cpp

1	#include <iostream>
2	#include <vector>
3	using namespace std;
4	
5	int main() {
6	int N;


```

7      cin >> N;
8
9      char vertex[N];
10     for (int i=0; i<N; i++) {
11         cin >> vertex[i];
12     }
13
14     int Graph[N][N];
15     for (int i=0; i<N; i++) {
16         for (int j=0; j<N; j++) {
17             cin >> Graph[i][j];
18         }
19     }
20
21     cout << "\nAdjacency List : \n";
22     for (int i=0; i<N; i++) {
23         cout << vertex[i] << " -> ";
24
25         bool hasEdge = false;
26         for (int j=0; j<N; j++) {
27             if (Graph[i][j] > 0) {
28                 if (hasEdge) cout << ", ";
29                 cout << "(" << vertex[j] << ", " <<
Graph[i][j] << ")";
30                 hasEdge = true;
31             }
32         }
33
34         if (!hasEdge) cout << "-";
35
36         cout << endl;
37     }
38 }

```

B. Output Program



```
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> g++ soal_2.cpp -o soal_2
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> .\soal_2.exe
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0

Adjacency List :
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> |
```

Gambar 16. Screenshot Jawaban Soal 2 Modul 6

C. Pembahasan

Kode berikut merupakan algoritma mengubah adjacency matrix menjadi adjacency list. Inputnya berupa N yaitu jumlah vertex dan adjacency matrix yang sudah berisi weighted edge. Outputnya adjacency list yang menyebutkan edge-edge dari vertex ini ke vertex itu.

Masuk ke fungsi main, cara kerjanya pertama-tama, menerima input nilai N dan nama-nama vertex ke array vertex. Lalu input adjacency matrix dengan ukuran NxN. Lalu loop i dari 0 sampai N untuk print adjacency list setiap vertex. Setiap baris, print nama vertex dan tanda panah. Lalu loop j dari 0 sampai N untuk edge setiap kolom. Kalau edge lebih dari 0, print nama vertex beserta weighted edge tersebut. hasEdge digunakan untuk mengeprint koma jika sebelumnya sudah ada edge yang di print. Jika vertex tidak memiliki edge, print strip (-).

Time complexity $O(n^2)$ karena input adjacency matrix dan print adjacency list. Space complexity $O(n^2)$ karena menyimpan vertex sebanyak N dan menyimpan adjacency matrix dari input.

SOAL 3

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS). Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1.

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9	16

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (7,9). Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel (2,1).

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari (0,0) ke (7,9) adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Tabel 23. Source Code File soal_3.cpp

```
1  #include <iostream>
2  #include <queue>
3  using namespace std;
4
5  struct Node {
6      int r, c, jarak;
7  };
8
9  int dr[] = {-1, 1, 0, 0};
10 int dc[] = {0, 0, -1, 1};
11
12 int main() {
13     int R, C, x_awal, y_awal, x_tuju, y_tuju;
14     cin >> R >> C;
15
16     int G[R][C];
17     for (int i=0; i<R; i++) {
18         for (int j=0; j<C; j++) {
19             cin >> G[i][j];
20         }
21     }
22     cin >> x_awal >> y_awal;
23     cin >> x_tuju >> y_tuju;
24
25     bool visited[R][C];
26     for (int i=0; i < R; i++) {
27         for (int j=0; j < C; j++) {
28             visited[i][j] = false;
29         }
30     }
31
32     queue<Node> q;
33
34     q.push({x_awal, y_awal, 0});
35     visited[x_awal][y_awal] = true;
36
37     int totalLangkah = -1;
38
39     while (!q.empty()) {
40         Node curr = q.front();
41         q.pop();
42
43         if (curr.r == x_tuju && curr.c == y_tuju) {
44             totalLangkah = curr.jarak;
```

```

45         break;
46     }
47
48     for (int i=0; i < 4; i++) {
49         int nr = curr.r + dr[i];
50         int nc = curr.c + dc[i];
51
52         if (nr >= 0 && nr < R && nc >= 0 && nc < C)
53     {
54         if (G[nr][nc] == 0 && !visited[nr][nc])
55     {
56         visited[nr][nc] = true;
57         q.push({nr, nc, curr.jarak + 1});
58     }
59     }
60
61     cout << totalLangkah;
62 }

```

B. Output Program



```

PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> g++ soal_3.cpp -o soal_3
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> .\soal_3.exe
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria>

```

Gambar 17. Screenshot Jawaban Soal 3 Modul 6

C. Pembahasan

Kode berikut merupakan algoritma BFS (Breadth-First Search), bekerja dengan visit tetangga (titik terdekat, mau itu atas bawah kiri kanan) sebelum melangkah lebih jauh. Gunanya untuk mencari jarak paling pendek untuk mencapai titik tujuan.

Pertama, ada struct `Node`, gunanya menyimpan posisi titik sekarang. Isinya `r` dan `c`, titik koordinat di baris sekian dan kolom sekian, serta jarak (jumlah langkah dari posisi awal hingga posisi ini). Lalu ada array global `dr` dan `dc`, distance row dan distance column. Menyimpan angka untuk perhitungan arah gerak, bisa ke atas, kanan, bawah, kiri.

Lalu masuk ke fungsi `main`. Cara kerjanya pertama input `R`, `C`, lalu graph `RxC` dan koordinat awal dan tujuan. Lalu inialisasi array `visited` seukuran `RxC` dan semuanya diisi `false`. Lalu inialisasi queue BFS, lalu push posisi awal `(0, 0)` dan jarak `0`. Lalu tandai itu sebagai `visited = true`. `totalLangkah` diisi `-1` kalau tujuan tidak bisa dicapai. Lalu loop BFS selama queue tidak kosong, ambil node paling depan, kalau koordinatnya sama dengan tujuan, simpan jaraknya ke `totalLangkah` dan break loop. Kalau belum sampai tujuan, loop `i 0` sampai `3` untuk mengecek titik yang cocok untuk dilewati. Misalnya node berada di `(0, 0)`, maka ia mengecek ke atas, kiri, kanan, bawah dahulu. Kalau titik itu ada di dalam grid, belum `visited`, dan nilainya `0` (bukan tembok), maka melangkah ke titik itu dan titik itu ditandai `visited = true`. Lalu di push ke queue dan jarak bertambah `1`. Setelah BFS selesai, print total langkah yang dilewati.

Time complexity $O(RxC)$, $((R \times C) \times 4)$ karena ada loop untuk membuat graph berukuran `RxC` dan while loop untuk jalan yang di dalamnya ada loop untuk menjelajahi titik juga, space complexity $O(RxC)$ karena perlu array 2D untuk graph itu.

SOAL 4

Time Limit: 2.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif. Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (4,5). Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel (0,5).

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Tabel 24. Source Code File soal_4.cpp

```

1  #include <iostream>
2  using namespace std;
3
4  int R, C, x_tuju, y_tuju;
5  int G[7][7];
6  bool visited[7][7];
7  int totalJalur = 0;
8
9  int dr[] = {-1, 1, 0, 0};
10 int dc[] = {0, 0, -1, 1};
11
12 void dfs(int r, int c) {
13     if (r == x_tuju && c == y_tuju) {
14         totalJalur++;
15         return;
16     }
17
18     for (int i=0; i<4; i++) {
19         int nr = r + dr[i];
20         int nc = c + dc[i];
21
22         if (nr >= 0 && nr < R && nc >= 0 && nc < C &&
G[nr][nc] == 0 && !visited[nr][nc]) {
23             visited[nr][nc] = true;
24             dfs(nr, nc);
25             visited[nr][nc] = false;
26         }
27     }
28 }
29
30
31 int main() {
32     cin >> R >> C;
33
34     for (int i=0; i<R; i++) {
35         for (int j=0; j<C; j++) {
36             cin >> G[i][j];
37         }
38     }
39     int x_awal, y_awal;

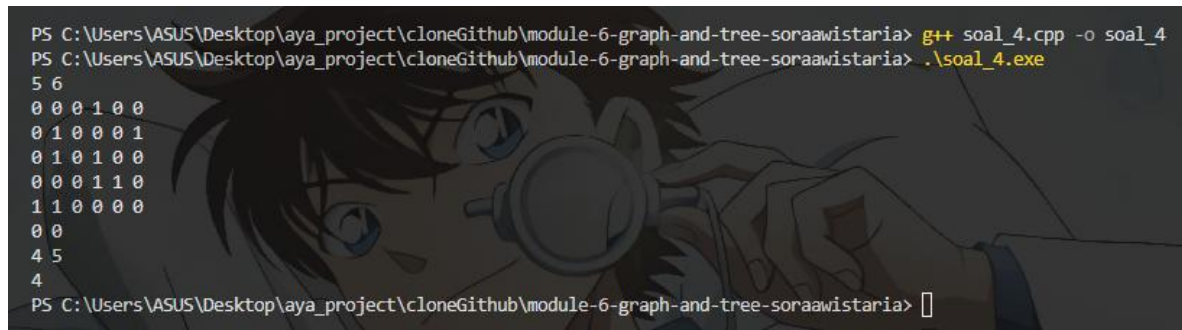
```

```

40     cin >> x_awal >> y_awal;
41     cin >> x_tuju >> y_tuju;
42
43     visited[x_awal][y_awal] = true;
44     dfs(x_awal, y_awal);
45
46     cout << totalJalur;
47 }

```

B. Output Program



```

PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> g++ soal_4.cpp -o soal_4
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> .\soal_4.exe
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> 

```

Gambar 18. Screenshot Jawaban Soal 4 Modul 6

C. Pembahasan

Kode berikut merupakan algoritma DFS (Depth-First Search), bekerja dengan menelusuri semua sampai titik tujuan. Algoritma ini memilih 1 jalur, lalu jalani jalur itu hingga sampai tujuan atau mentok nabrak jika tujuan tidak sampai. Cara kerjanya dengan implementasi stack dan fungsi rekursif.

Pertama, ada variabel global, agar variabel ini bisa digunakan di fungsi main dan fungsi DFS. Variabel globalnya yaitu R dan C untuk ukuran grid graph, x_tuju dan y_tuju untuk koordinat tujuan, G[7][7] untuk membuat grid (ukuran 7 sesuai constraint), visited[7][7] untuk menyimpan jalur yang sudah dikunjungi, dan total jalur untuk menyimpan berapa jalur yang bisa dipakai untuk sampai tujuan. Lalu array dr dan dc untuk menyimpan hitungan 4 arah gerak, atas bawah kiri kanan.

Kedua, fungsi dfs untuk mencoba semua jalur yang mungkin, dari titik awal sampai titik tujuan. Cara kerjanya yaitu membuat base case dulu, kalau posisi sekarang sudah sama dengan titik koordinat tujuan, total jalur ditambah 1 lalu return. Kalau belum sampai tujuan, cek atas bawah kiri kanan. Titik yang valid (tidak diluar grid, nilainya 0, dan belum visited)

akan dilangkah dan ditandai `visited = true`. Lalu rekursi dari titik sekarang. Setelah rekursi selesai, titik tadi ditandai `visited = false` agar jalur bisa dipakai lagi untuk eksplorasi cabang lain. Time complexity-nya $O(4^{(RxC)})$ karena setiap titik bisa bercabang ke 4 arah, mencoba ke 4 arah tersebut sebanyak ukuran grid graph. Sedangkan space complexity-nya $O(RxC)$ karena menyimpan grid graph seukuran RxC dan call stack yang bisa mencapai RxC jika worst case (misalnya tidak ada obstacle, semua grid nilainya 0).

Terakhir, fungsi `main`. Dimulai dari input R dan C , lalu input grid graph, lalu input koordinat awal dan tujuan. Lalu posisi awal ditandai `visited = true` agar tidak balik ke koordinat awal selama proses DFS. Lalu panggil fungsi DFS dari posisi awal, lalu print total jalur. Time complexity-nya $O(RxC + 4^{(RxC)})$ untuk input grid + fungsi dfs berjalan, space complexity $O(RxC)$ karena memori untuk grid graph, titik visited, dan call stack.

SOAL 5

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah Red-Black Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree. Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N$

Q

T_1

T_2

...

T_Q

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu

Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-10000000000 \leq A_i \leq 10000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder] : 20 40 30 60 80 70 50
ALL	

Penjelasan Contoh

Bilangan dimasukkan ke RBTree secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTree yang dibentuk sudah konsisten dengan aturan RBTree.

A. Source Code

Tabel 25. Source Code File soal_5.cpp

1	#include "RedBlackTree.h"
2	#include <iostream>
3	#include <vector>
4	using namespace std;

```

5
6 void preorder (const RedBlackTree::Node* node, const
  RedBlackTree::Node* nil, vector<int>& hasil) {
7     if (node->isNil) return;
8     hasil.push_back(node->key);
9     preorder(node->left, nil, hasil);
10    preorder(node->right, nil, hasil);
11 }
12
13 void inorder (const RedBlackTree::Node* node, const
  RedBlackTree::Node* nil, vector<int>& hasil) {
14     if (node->isNil) return;
15     inorder(node->left, nil, hasil);
16     hasil.push_back(node->key);
17     inorder(node->right, nil, hasil);
18 }
19
20 void postorder(const RedBlackTree::Node* node, const
  RedBlackTree::Node* nil, vector<int>& hasil) {
21     if (node->isNil) return;
22     postorder(node->left, nil, hasil);
23     postorder(node->right, nil, hasil);
24     hasil.push_back(node->key);
25 }
26
27 void print(string label, vector<int>& hasil) {
28     cout << label << " : ";
29     for (int i=0; i< hasil.size(); i++) {
30         if (i > 0) cout << " ";
31         cout << hasil[i];
32     }
33     cout << endl;
34 }
35
36 int main() {
37     RedBlackTree rbt;
38
39     int N;
40     cin >> N;
41
42     for (int i=0; i<N; i++) {
43         int num;
44         cin >> num;
45         if (!rbt.contains(num)) {
46             rbt.insert(num);
47         }
48     }

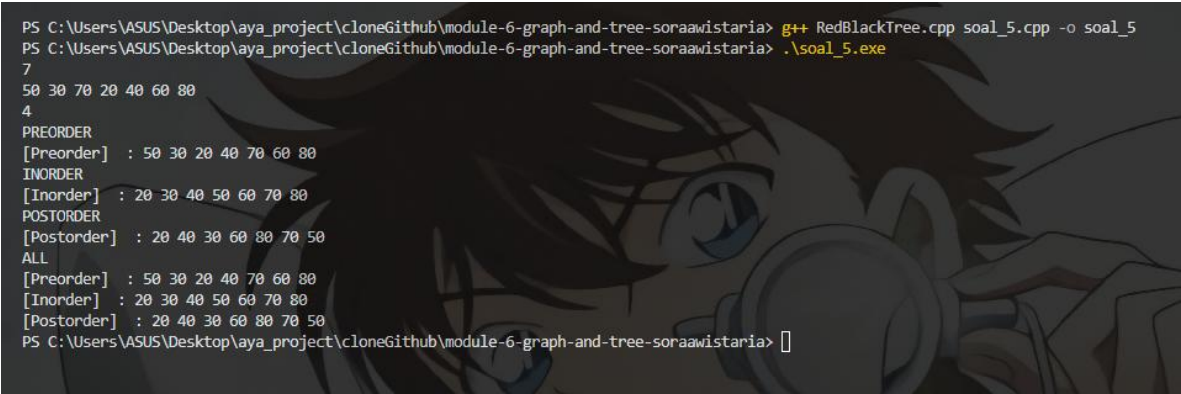
```

```

49
50     int Q;
51     cin >> Q;
52
53     for (int i=0; i<Q; i++) {
54         string query;
55         cin >> query;
56
57         if (rbt.empty()) {
58             cout << "Tree kosong. Tidak ada yang bisa
ditampilkan.\n";
59             continue;
60         }
61
62         vector<int> pre, in, post;
63         preorder(rbt.root(), rbt.nil(), pre);
64         inorder(rbt.root(), rbt.nil(), in);
65         postorder(rbt.root(), rbt.nil(), post);
66
67         if (query == "PREORDER") {
68             print("[Preorder] ", pre);
69         } else if (query == "INORDER") {
70             print("[Inorder] ", in);
71         } else if (query == "POSTORDER") {
72             print("[Postorder] ", post);
73         } else if (query == "ALL") {
74             print("[Preorder] ", pre);
75             print("[Inorder] ", in);
76             print("[Postorder] ", post);
77         }
78     }
79 }

```

B. Output Program



```

PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> g++ RedBlackTree.cpp soal_5.cpp -o soal_5
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> .\soal_5.exe
7
50 30 70 20 40 60 80
4
PREORDER
[Preorder] : 50 30 20 40 70 60 80
INORDER
[Inorder] : 20 30 40 50 60 70 80
POSTORDER
[Postorder] : 20 40 30 60 80 70 50
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> 

```

Gambar 19. Screenshot Jawaban Soal 5 Modul 6

C. Pembahasan

Kode berikut merupakan algoritma untuk traverse struktur data tree dengan preorder, inorder, maupun postorder.

Fungsi `preorder`, untuk traverse dari root ke kiri lalu ke kanan. Cara kerjanya, membuat base case jika node sekarang adalah nil node maka langsung return. Kalau bukan, simpan (`push`) node sekarang ke vector `hasil`. Lalu rekursif ke kiri, rekursif lagi ke kanan. Time complexity $O(n)$ karena setiap node di visit sekali, space complexity $O(n)$ karena menyimpan nilai node ke vector dan call stack untuk fungsi rekursi.

Fungsi `inorder`, untuk traverse dari kiri ke root lalu ke kanan. Cara kerjanya, membuat base case sama seperti preorder. Kalau bukan nil, rekursif ke kiri, `push` nilai `key` ke vector `hasil`, lalu rekursif lagi ke kanan. Time complexity $O(n)$ karena setiap node di visit sekali, space complexity $O(n)$ karena menyimpan nilai node ke vector dan call stack untuk fungsi rekursi.

Fungsi `postorder`, untuk traverse kiri ke kanan lalu ke root. Cara kerjanya, membuat base case sama seperti preorder. Kalau bukan nil, rekursif kiri lalu rekursif kanan. Lalu `push` `key` ke vector `hasil`. Time complexity $O(n)$ karena setiap node di visit sekali, space complexity $O(n)$ karena menyimpan nilai node ke vector dan call stack untuk fungsi rekursi. Fungsi `print`, untuk print hasil traverse masing-masing jenis traversal. Cara kerjanya, print label alias jenis order, lalu loop vector `hasil` dan print setiap elemennya. Time complexity untuk mengeprint seisi vector $O(n)$ sedangkan space complexity $O(1)$ karena tidak menggunakan memori baru.

Fungsi `main`, cara kerjanya inisialisasi `RedBlackTree` dahulu. Lalu menerima input `N` dan setiap angka untuk dibuat tree. Setiap angka di cek, agar tidak ada angka yang duplikat. Kalau tidak duplikat, `insert` ke tree. Lalu input `Q` untuk berapa kali query dan querynya apa. Setiap iterasi di cek dulu apakah tree kosong atau tidak. Kalau kosong, print pesan dan `continue` ke query berikutnya. Kalau tree tidak kosong, traversal tree dengan ketiga jenis traverse-nya. Hasil traverse di print sesuai query, mau itu preorder, inorder, postorder, maupun all (ketiganya). Time complexity-nya $O(n \log n + Q \times N)$ karena insert elemen ke RBT $O(\log n)$ dan query traversal $O(n)$. Space complexity $O(n)$ karena menyimpan tree dan vector traversal.

TAUTAN GIT

https://github.com/soraawistaria/Praktikum_AlgoDS